How Is Behavioral Equivalence Evaluated in LLM-Based Code Translation and Migration? A Systematic Mapping Study

Author Name 1
Institute 1
City, Country
author1@example.com

Author Name 2
Institute 2
City, Country
author2@example.com

Author Name 3
Institute 3
City, Country
author3@example.com

Author Name 4
Institute 4
City, Country
author4@example.com

Author Name 5
Institute 5
City, Country
author5@example.com

Abstract

[P01, Major; EIC, DA; open since R01: Pink notes are the 30/10 review by a five-reviewer panel plus deep research; reasons, evidence and the revision plan are in Review_Claude_Seminar_2_3010.md at the repository root. Panel decision: major revision. Before any clean build: setting reviewfalse in macros.tex still fails (Undefined control sequence hcrev, about 100 errors), because hcrev and revtag are defined only in the review branch; add pass-through definitions to the else branch. Both builds still print the placeholder author block.**]** Large language models (LLMs) are now used to translate code between languages and to migrate it across versions, libraries and frameworks.**[P02, Critical for EASE; EIC:** The EASE build is 18 pages (NeurIPS build 28) against a 10-page limit; over-length papers are desk-rejected. Cut plan (EIC-01 in the report): move the Appendix table and Tables 2, 4 and 6 and the track and year rows of Table 8 to the replication package; Discussion from about 3.4k to 1.3k words; pilot to about 800 words; this abstract from about 340 words and 20 numbers to 200–250 words. If it still does not fit, publish the pilot separately.**]** Whether the generated code behaves like the original is decided by an oracle, and the choice of oracle can change the reported result. We present a systematic mapping study of how behavioral equivalence is evaluated in LLM-based code translation and migration,**[P03, Minor; EIC, R2:** The abstract never says what is new compared with the existing reviews (Chen et al., Zhang et al., Rgaguena et al.). Add one clause, e.g. prior reviews catalogue translation metrics but do not trace where test ground truth comes from or how far oracles disagree on the same outputs.**]** covering the main proceedings of ICSE, FSE and ASE from 2024 to 2026.**[P04, Minor; R1, DR; open since R02, R10:** Main proceedings here also covers Companion-volume papers (some from workshops), posters and journal-first talks (Section 4.1), and refs.bib gives seven FSE Companion papers main-proceedings booktitles. Most readers will take it to mean the research-track volume; say proceedings (main and Companion volumes).**]** From 4,451 papers we selected 49 and coded each from its full text. The most common oracles are compilation (71%) and existing tests (57%); formal verification, LLM-generated tests and LLM judges are rare (4% each).**[P05, Major; R2, DA, R1; open since R08:** Two qualifiers. (1) Compilation 71% counts any reported compile count, including failure breakdowns (Section 4.5), and 33 of these 35 papers also run the code (all but CodePlan and DepUpgrade), so compilation is almost never the only evidence; say so. (2) LLM-generated tests 4% undercounts LLM-written tests: O-e is defined by where expected outputs come from, and 3 more papers run LLM-generated inputs under O-f. See the Table 1 comment.**]** Only 29% of the papers run the original program to compare behavior**[P06, Major; DA, R2:** A project suite that passed on the pre-migration code already compares against the original's behavior on its inputs, so 29% run the original is not the same as 71% never compare with the original. Reword (e.g. only 29% run the original program on the same inputs as the migrated code) and see the G5 comment.**]**, and 18% use no oracle that executes the code. In translation between languages, existing tests come mostly from benchmarks; in library and version migration, they usually come from the project's own suite. None of the 5 papers on legacy migration uses such tests**[P07, Major; EIC, R2, R1, DA:** This depends on a legacy flag that excludes C, although at least 4 mapped C-to-Rust papers call their sources legacy and use existing tests; all 5 papers are also industry or NIER papers, and legacy systems often have no automated tests. Drop it from the abstract or qualify it (5 papers with COBOL, PL/I or mainframe sources). See the comment on the legacy flag in Section 2.**]**, and 9 of the 12 on library migration include a human check. Thirty-three papers report separate results from two or more oracles on the same outputs. In 13 of the 18 that give both a compile rate and a behavioral rate, the compile rate is more than 10 percentage points higher.**[P08, Major; R1, DA, R2, EIC; open since R21:** 13 of 18 depends on which configuration represents each paper (no rule is stated; 4 rows are baselines, an ablation or pre-repair output) and on a 10-point cut that is not justified. See the comment in Section 5.4; also give the median gap (19 points).**]** Yet 10 of the 49 papers state no limitation of their own oracle.**[P09, Minor; DA, EIC:** 5 of these 10 are short papers (IVR, two posters, a demo, NIER), i.e. 5 of the 7 short papers against 5 of the 42 full papers, so page limits plausibly explain half. Yet also links two unrelated findings. Drop Yet and add half of them short papers; note it in Threats and next to checklist item 7.**]** In a pilot that varied only the oracle, we judged an LLM's translations of 164 HumanEval-X tasks from Python to Java with four oracles: compilation, the benchmark tests, CodeBLEU and a second LLM as judge. Across oracles and thresholds, 52% to 90% of the same translations count as correct.**[P10, Major; EIC, R1, DA; open since R04:** Both ends are weak anchors: 90% is compilation, which no mapped paper treats as correctness, and 52% is CodeBLEU at 0.7, a cut-off you chose and do not treat as main (67% at 0.6). The mapped papers report CodeBLEU as an average,

not as a pass/fail verdict. Suggest: between 67% (CodeBLEU at the main threshold) and 90% (compilation) of the same translations are accepted, against 80% under the tests; and name the setup (a 7B translator, one run). The same sentence is in the Introduction and the Conclusion.] The judge accepts half of those that fail the tests and rejects 41% of the benchmark's own reference solutions.**[P11, Major; DA, R1, R2, EIC; open since R04:** Say a small (8B, 4-bit) LLM judge, as the Conclusion does. The 41% is not yet judge error: the references are not proven equivalent to the Python code and none of the 67 rejected ones was inspected; a judge asked about every valid input can rightly reject, e.g., an int-overflow difference. See the pilot comments.] A success rate is thus meaningful only together with the oracle that produced it. From these results we derive five gaps between current evaluation practice and the needs of real migration projects**[P12, Major; DA, R3, EIC, R1:** The needs side rests on one premise argued in Section 6 (a migration replaces an existing program, so it needs evidence...). No practitioner data, acceptance criteria or V&V standard is analysed, and the map's own industry papers partly cut against it. Reword here, in RQ5, contribution (5) and Section 6, e.g. gaps between how published studies evaluate migrations and the evidence that behavioral equivalence requires; the Conclusion's what a migration needs is already closer.], along with a checklist for reporting oracle provenance. We release our data and code.**[P13, Major; EIC, R1:** The release tag is still a placeholder and, per the README, the repository has no release or license yet. Several printed counts come from reading rather than from coded fields (18 papers classify failures, 5 describe faults that slip past one check, the G4 cases), so the package cannot recompute them until reading_counts.csv is added. Create a tagged, licensed release archived with a DOI before submission; for EASE use an anonymized mirror.]

Keywords

code translation, code migration, large language models, test oracle, behavioral equivalence, systematic mapping study

1 Introduction

Code migration is routine industrial work, and large language models (LLMs) now take on part of it. Google reports LLM-assisted migrations across its monorepo, such as JUnit 3 to JUnit 4 [40]. LLM-based approaches also target C-to-Rust ports [53], COBOL modernization [16] and dependency upgrades [35]. In every case someone must decide whether the migrated code still does what the original did. That decision rests on an *oracle*: a compiler, a test suite, a similarity metric, a comparison with the original program, a formal check, a human or another LLM.

The oracle is not neutral. Published studies report very different success rates for the same outputs, depending on the oracle:

- DepWareTrans [8]: on an industrial repository, file-by-file Java-to-Kotlin translation with iterative feedback reaches a 38.16% compile rate but only a 9.39% test pass rate.**[P14, Major; R2, DA, EIC; open since R05:** These numbers are the file-by-file baseline (STAR) that DepWareTrans improves on, as the Figure 3 caption says. Per R2, DepWareTrans itself reports 100% compilation and test success (arXiv 2608.14128), with compile- and test-driven validation in its loop, which is itself a G4 case. As written, readers attribute 9.39% to the proposed system. Write a file-by-file baseline in the DepWareTrans study, or replace the example.]
- DepTrans [14]: in C-to-Rust translation, a variant trained with a compile-only reward reaches a 97.9% compile rate but a 0% test pass rate.**[P15, Major; R2, DA, EIC:** This is an ablation rewarded only for compiling, built to show reward hacking (the model emits panic!), and it is the 98-point extreme of Figure 3: it shows oracle gaming, not two oracles disagreeing on realistic outputs. Use a main configuration (e.g. VERT, 94.7% compile vs 51.9% property-based tests, or DepTrans's full system, 60.7% vs 43.5%) and move the ablation to the oracle-gaming paragraph. All three examples also compare compilation with a behavioral check, which are nested (code must compile to pass); add one non-nested example from Table 9 (e.g. TerraMod, similarity vs execution).]
- DepUpgrade [35]: dependency upgrades compile in 119 of 123 tasks (96.7%), but reviewers judge only 62 of these 119 patches (52.1%) semantically equivalent.

We observe the same in a pilot that applies four oracles to one fixed set of 164 LLM translations (Section 6). Across oracles and thresholds, 52% to 90% of the translations count as correct. A reported success rate therefore depends on the oracle as well as on the system under evaluation. Even tests can be a weak oracle. Outside migration, existing regression tests catch only 7.6% of the semantics-changing code changes studied by Gröninger et al. [20].**[P16, Minor; R2, DR:** The figure is correct (10 of 131 changes), but the low recall partly reflects changes that no test covers or that cannot be built; add partly because many of these changes are not covered by any test.]

These examples raise two questions: which oracles does research on LLM-based migration use, and how far can their verdicts be trusted? Reviews that cover code translation examine its techniques, evaluation metrics, publication trends and security issues (Section 3). One recent survey also orders evaluation signals into six levels and discusses how reliable each is [62]. None of these reviews traces where the ground truth of test-based oracles comes from, quantifies how far oracles disagree on the same outputs or compares oracle use across contexts. Our systematic mapping study of the main proceedings of ICSE, FSE and ASE from 2024 to 2026 addresses all three.**[P17, Minor; EIC, R1, DA:** The title and RQ5 read as field-wide, while the evidence is 3 venues over 3 years (49 papers) plus a single-run pilot. Consider putting the venues and the pilot in the title, e.g. ... at ICSE, FSE and ASE 2024–2026, with a Four-Oracle Pilot.] The map answers RQ1 to RQ3 and the pilot answers RQ4; RQ5 draws on both:

RQ1. Which oracles are used to evaluate behavioral equivalence in LLM-based code translation and migration?

RQ2. Where do test-based oracles get their ground truth, and how does oracle use differ by context?

RQ3. Which threats to oracle reliability do the studies acknowledge or demonstrate?

RQ4. How do automatic oracles that do not execute the code compare with existing tests on the same LLM translations?

RQ5. What gaps separate current evaluation practice from the needs of real migration projects, and what should studies report and investigate next?

Our contributions are (1) a coded map of 49 primary studies by oracle, ground-truth origin and context of use, including the combinations that no study covers**[P18, Minor; DA:** Results says each empty cell would hold at most 1.3 papers by chance and that an empty cell says little on its own, so presenting empty combinations as a contribution conflicts with it. Reword, e.g. combinations that no study in the map covers, most of them uninformative at this sample size.**]**; (2) a catalog of the 33 studies that report separate results from two or more oracles on the same outputs, showing how far the oracles disagree in the 24 whose numbers can be compared; (3) a classification of the limitations that studies report about their own oracles; (4) an open harness that applies four oracles to the same LLM translations, used in a pilot on 164 HumanEval-X tasks**[P19, Major; EIC, DA, R2:** Position the pilot against what is known: similarity vs execution (TransCoder introduced computational accuracy because BLEU misleads, Roziere et al. 2020; Evtikhiev et al., JSS 2023), weak benchmark tests (EvalPlus, NeurIPS 2023) and LLM judges for code (Wang et al., ISSTA 2025; CodeJudge; ICE-Score). Without them the pilot reads as new evidence rather than a replication in a translation setting. Its genuinely new parts are the known-answer check of the judge and the mutation check that exposes test cases lost in porting (HumanEval/131); say so in one sentence here and in Related Work.**]**; and (5) five gaps between current evaluation practice and the needs of real migration projects, along with a checklist for reporting oracle provenance. We release our data and code (Section 9).

Structure. Section 2 defines the migration kinds and oracle categories we code. After a review of related studies (Section 3), Section 4 describes the search, selection and extraction. Results for RQ1 to RQ3 follow in Section 5, and the pilot answers RQ4 in Section 6. Section 7 answers RQ5 and proposes the checklist. We close with threats to validity (Section 8) and conclusions (Section 9). Appendix A lists every included paper with its coding.

2 Background

Kinds of migration. We distinguish five kinds of migration: between programming languages (XL, e.g. C to Rust or COBOL to Java); between versions of a language or platform (VER, e.g. Java 8 to 17); between libraries or APIs, including dependency upgrades with breaking changes (LIB, e.g. JUnit 3 to 4); between dialects (DIA, e.g. PostgreSQL to MySQL); and between frameworks or application platforms (FW, e.g. Android XML views to SwiftUI**[P20, Minor; R2; open since R07:** SwiftUI also changes language and platform (to Swift on iOS), so by your own definitions this is also XL; GUIMigrator is coded FW only while IP-preserving is FW/XL, and DeepModelX (deep-learning frameworks) is FW although such frameworks are usually treated as libraries. Use an unambiguous example (Android XML views to Jetpack Compose, AngularJS to Angular) and state when FW also counts as XL and how FW differs from LIB.**]**). We use migration as the umbrella term and call the XL and DIA kinds translation, because they rewrite code in another language or dialect. A study may involve several kinds. Independently of the kind, a *legacy* flag marks a source written for a language or platform that is no longer in mainstream use for new development, such as COBOL, PL/I, mainframe systems or ASP.NET Web Forms. The flag follows the source technology, not the authors' description of their code.**[P21, Major; R2, R3, EIC:** This departs from the SE literature (legacy systems are old, business-critical systems that resist change, whatever the language; Khadka et al., ICSE 2014) and from how 4 or more mapped C-to-Rust papers describe their sources, and an abstract-level finding depends on it. What constrains oracle choice is the system's condition (can the original run with its environment, do tests exist), not the language. Rename the flag to what it measures (e.g. obsolete-platform source) or code legacy as the studies describe it and report both.**]**

Behavioral equivalence. Two programs are behaviorally equivalent if they produce the same observable behavior for every input: the same outputs; the same effects on files, databases or other state; and the same termination or failure.**[P22, Minor; R3:** This fits deterministic, single-call programs. Add that for stateful systems behavior is defined over input sequences, for nondeterministic or concurrent programs (the corpus has a concurrent Java-to-ArkTS converter) over the set of possible behaviors, and over valid inputs (as the pilot's judge prompt says); and state that non-functional properties (latency, batch windows, memory) lie outside the definition and were not coded.**]** Checking this exactly is undecidable in general, so every practical check approximates it. Tests and differential runs sample the input space; bounded model checking and symbolic execution cover it up to a bound; formal proofs cover it completely but only for a model of the program; and compilation, similarity metrics and human or LLM judgment do not execute the program. Migration also relaxes this definition in ways that an evaluation should make explicit. A translation may map types whose ranges differ, such as 32-bit integers in C into Python's unbounded integers; a library migration may keep the functional result but change logging, performance or error messages; and a framework migration may be judged on what a user sees rather than on internal state. "Equivalent" therefore always means equivalent with respect to some oracle and some notion of observable behavior.

Oracles. An oracle is the check a study uses to decide whether the migrated program behaves like the original. We use nine categories (Table 1).

The four test-based categories (O-c to O-f) differ in the origin of their ground truth, the expected outputs: a test suite that existed before the study (O-c); the original program's tests, translated for the study (O-d); an LLM or a specification (O-e); or a run of the original program during the study (O-f). Tests that an LLM writes therefore fall under O-f when their expected outputs come from running the original, because the LLM then supplies only the inputs.

The categories are not exclusive. Many studies combine several, and some use one inside the technique (for example to filter training data or to drive a repair loop) and another to report results. Section 4.5 states which of these uses we code.

3 Related Work

Reviews of code translation. Chen et al.'s systematic literature review of neural code translation analyzes 57 primary studies from seven perspectives and counts how many of them use the most

Table 1: Oracle categories used throughout the paper.

Code	Oracle	What it checks	Known weakness
O-a	Surface similarity (BLEU, CodeBLEU, exact match, AST match)	Closeness to a reference translation	Equivalent code can look different, and similar code can behave differently
O-b	Compilation, type checking or another well-formedness check	The output is well-formed in the target	Does not check behavior
O-c	Existing tests	Behavior on a test suite that existed before the study, built for a benchmark or by the project's developers	Only as strong as the tests; benchmarks often have few
O-d	Translated tests	Behavior on the original program's own tests after they are translated with the code; in test migration, on the migrated tests themselves	Test and code may share the same translation error; a migrated test that passes may no longer check what the original did
O-e	LLM-generated tests	Behavior on new tests whose expected outputs come from an LLM or a specification**[P23, Major; R2; open since R08:** Label and definition disagree: the label names who writes the tests, the definition where the expected outputs come from. LLM-written tests checked against the original are O-f (3 papers), and IP-preserving's tests come from a specification. The other O-e paper, TransAgent, is O-e only by the no-stated-source default; per R2, its arXiv benchmark section says it executes the collected human solutions on test inputs generated with gpt-4o-mini, which would make it O-c or O-f and leave no paper whose expected outputs come from an LLM. Re-read TransAgent; rename O-e (e.g. tests with LLM- or specification-derived expected outputs) here, in Figure 2 and in the abstract; report how many papers have an LLM write tests or inputs (O-e plus f:L).**]**	Expected outputs can be wrong or share a source with the output under test
O-f	Differential testing	Same outputs as the original program, run by the study on the same inputs (from fuzzing, existing tests or an LLM)	Limited by input generation; needs the original to run
O-g	Formal verification	Equivalence proven or bounded-checked against the original or a specification	Costly; bounds and models limit coverage
O-h	LLM judge	An LLM's verdict on equivalence	The judge's own accuracy is rarely validated**[P24, Minor; R2, DR:** True of the 2 mapped papers, not of LLM judges in general: Wang et al. (ISSTA 2025) find output-based judges reach a Pearson correlation of about 0.81 with human scores on code translation (ChrF++ about 0.34); PolyHuman and Moumoula et al. also evaluate equivalence judgments. Scope it (in the studies we map) and cite this work in Related Work and G2.**]**
O-i	Human evaluation	A person's judgment	Subjective, small samples, rarely agreement-measured

common evaluation metrics [9]. A later survey by Zhang et al. follows PRISMA to review 47 methods from 2013 to 2026 and orders evaluation signals into six levels, from text matching to testing the translation inside its project [62]. It discusses how reliable each level is but does not count the methods that use it. In a systematic mapping study, Rgaguena et al. classify 53 studies of generative AI for code translation by publication year and channel, research type, publication type, empirical study type, AI model and dataset [49]. The security-focused systematization of knowledge of AI4Code by Wu et al. catalogs the functional verification techniques used for translation without measuring their reliability [54]. These reviews cover translation between programming languages, whereas our map also includes library, version and framework migration.**[P25, Minor; R2, DR:** Qualify: Lamothe, Gueheneuc and Shang (ACM CSUR 2021) review API evolution, the library and version half of your scope, and Hou et al. (TOSEM 2024) review LLMs for SE

including translation. Say what the map adds beyond them.] It further addresses three questions that the reviews leave open: where the ground truth of test-based oracles comes from, how far oracles disagree on the same outputs and how oracle use differs between legacy and non-legacy sources or between kinds of migration.**[P26, Major; R2, EIC, DA:** Each gap is known individually; say what was known so the novelty (prevalence, provenance and context across 49 studies) is claimed precisely. G1: Roziere et al. 2020, Evtikhiev et al. (JSS 2023), CrystalBLEU (ASE 2022). G3: EvalPlus (NeurIPS 2023); mutation-based test adequacy of code benchmarks (JSEP 2025). G5: FLOURINE, Zhang et al. (PLDI 2025), Syzygy, LLMLift. Checklist: How2Bench (arXiv 2501.10711). Most directly, refs.bib has Khanna 2026, Passing Parity Is Not Preserving Behaviour: Oracle Escape in Function-Level LLM Code Migration (AISM workshop), whose title states your thesis, but it is never cited; cite and differentiate it.]

Evaluation of code translation. Jiao et al. classify translation tasks into four types by complexity and build a benchmark on which they evaluate mainly pre-LLM models [28]. They also find few unit tests and unverified reference translations in the widely used TransCoder-test.**[P27, Minor; DR, R2:** Overstated: Jiao et al. report that about half of the translation pairs have unit tests and that the reference translations were not verified by tests (arXiv 2308.08961). Write only about half of its pairs have unit tests.] On their own benchmark, test pass rates and similarity scores rank two LLMs differently. Whereas they compare metrics within one benchmark, we map evaluation practice across 49 studies from the LLM era, including LLM-generated tests and LLM judges.

The oracle problem. Barr et al. survey the test oracle problem in general [5].**[P28, Minor; R2, R3, DR:** Add the LLM-era update (Molina, Gorla and d'Amorim, Test Oracle Automation in the Era of LLMs, TOSEM 2025) and the classic notions your categories rest on: comparison with the original is a pseudo-oracle (Davis and Weyuker 1981) and differential testing (McKeeman 1998); compiler research on untrusted translators offers translation validation (Pnueli et al. 1998) and equivalence modulo inputs (Le et al., PLDI 2014). Mapping O-a to O-i onto Barr's classes would also show completeness; metamorphic oracles have no category.] Migration is a special case in which the original program is available as a reference, so differential testing and equivalence checking become possible. RQ1 counts how many of the studies in our map execute the original program to compare behavior (Section 5.2).

Individual studies. Several benchmarks and techniques address parts of the oracle problem: FreshBrew accepts a migration only if test coverage falls by at most 5 percentage points, which exposes reward hacking [37];**[P29, Minor; open since R09:** The coverage rule guards against reward hacking and, per Section 5.4, catches only part of it; which guards against reward hacking says what is meant. Coverage is also only weakly tied to fault detection (Inozemtseva and Holmes, ICSE 2014).] AlphaTrans translates the tests together with the code [25]; Dristi and Dwyer check equivalence by differential fuzzing without predefined tests [11]; and Sun et al. test whether LLMs can judge functional equivalence across languages on their PolyHuman dataset [51]. These works document defenses and failure modes one study at a time; RQ3 classifies such threats across the 49 studies in our map. FreshBrew and AlphaTrans are among the 49. The other two are not: Dristi and Dwyer study refactoring within a single language, and Sun et al. published at ESEM.

4 Method

This section describes how we built and coded the map that answers RQ1 to RQ3. The pilot for RQ4 has its own design (Section 6), and RQ5 draws on both (Section 7). We followed the guidelines for systematic mapping studies in software engineering [45]. The protocol, search log, codebook and all decisions are in the replication package.

4.1 Scope

A paper is in scope if it appears in the 2024, 2025 or 2026 edition of ICSE, FSE or ASE: in any track of the conference program, including the poster, doctoral symposium and student research competition tracks; in the conference's main or Companion volume; or, for journal-first papers, as a presentation at the conference. We call this set the main proceedings, as opposed to co-located events with proceedings of their own. ICSE, FSE and ASE were selected as major general software engineering conferences with both research and industry tracks.**[P30, Major; R1, EIC, DA:** This does not explain why ISSTA, the flagship testing venue where oracle work concentrates, is excluded in 2024 and 2026, when it was not co-located, or why TSE, TOSEM and EMSE count only as journal-first. Venue choice can bias exactly the RQ1 shares the paper stresses (formal 4%, differential 29%). Scan ISSTA 2024–2026 with the frozen string as a sensitivity set, or give a substantive reason and scope the title and general statements to the three venues.] We examine the evaluation practices reported in their peer-reviewed papers, including studies of industrial and legacy-system migrations. Co-located workshops and conferences that publish their own proceedings (ReCode, LLM4Code, FORGE, AGENT, AIware, AISM, ICPC, MSR and others) are out of scope: the set of events changes from year to year, so counting them would make years incomparable.**[P31, Minor; R1; open since R10:** Workshops that publish in the Companion volume also change from year to year (Companion-only papers: FSE 4, 74 and 95 in 2024–2026; ASE none), so the same argument applies to them, and they sit in the yearly denominators of Section 5.1. Exclude them and recompute, or justify keeping them and report their number per year.] ISSTA 2025 and Internetware 2025, which were co-hosted with FSE that year only, are excluded for the same reason. Related papers outside the corpus are cited in Sections 1, 3 and 7 but not counted.

4.2 Search

We applied one frozen keyword string to titles and abstracts:

```
translat* OR transpil* OR migrat* OR moderni* OR legacy
OR porting OR ported OR upgrad* OR deprecat* OR
"cross-language" OR "cross-lingual" OR equivalen*
```

The string omits "oracle" and "behavior*", because they pulled in unrelated testing papers, and omits "LLM", because many neural papers do not use the word; whether a paper uses a neural model or an LLM is checked at screening (Section 4.3). Paper lists came from each conference program on conf.researchr.org, covering every track that had a published paper list (the FSE 2026 and ASE 2026

Table 2: [new] Search and selection by conference and year. Papers: main-proceedings papers scanned; No abstract: papers matched on their title only; Hits: keyword hits; Candidates: hits plus the one known paper the search missed; Included: main corpus; I3: papers in the separate cross-language equivalence-detection tier (Section 4.3).

Venue	Papers	No abstract	Hits	Candidates	Included	I3
ICSE 2024	611	61	58	58	2	0
ICSE 2025	526	0	41	41	4	0
ICSE 2026	796	1	68	68	7	0
FSE 2024	261	6	17	17	2	0
FSE 2025	451	1	35	36	6	1
FSE 2026	563	2	64	64	11	1
ASE 2024	299	1	25	25	4	0
ASE 2025	447	1	53	53	4	0
ASE 2026	497	7	69	69	9	0
Total	4,451	80	430	431	49	2

Table 3: Inclusion (I) and exclusion (E) criteria.

Code	Criterion
I1	Uses a neural model or LLM to translate or migrate code between a named source and target (language, version, dialect, library or API, framework) and describes how equivalence is checked, even if only by BLEU
I2	Proposes or studies a method, benchmark or oracle to evaluate translations or migrations
I3	Detects equivalence between programs in two different languages; reported as a separate tier
I4	English, peer-reviewed
E1	Natural-language translation, or NL-to-code tasks with no code-to-code translation
E2	No information at all on how outputs are evaluated
E3	Clone or similarity detection that neither targets functional equivalence nor relates to translation
E4	Secondary study
E5	Duplicate of the same work (we keep the most complete version)
E6	Full text unavailable

poster tracks did not; Section 8). We cross-checked them against the main and Companion volumes of each conference through Crossref, which added 216 papers that appear in no track program. Some of them come from workshops that publish in the Companion volume, and none of their 13 keyword hits was included. Missing abstracts were taken from Semantic Scholar and arXiv. The final scan, on October 23, 2026, covered 4,451 main-proceedings papers and returned 430 keyword hits; for 80 papers without an available abstract, the string was matched against the title only.**[P32, Minor; R1:** Publisher pages (ACM DL, IEEE Xplore) normally have these abstracts. Fetch the 80, rerun the string and screen any new hits; this removes the limitation instead of discussing it, including its effect on the 2024 share in Section 5.1.**]** Table 2 gives the counts for each conference and year. Most papers without an abstract are from ICSE 2024 (61 of 80).

To estimate recall, we used 40 in-scope papers that meet our criteria and that we had identified before the scan, in exploratory searches of the same proceedings and while reading related work. The string retrieved 39 of the 40 (97.5%).**[P33, Major; R1, R2, DR; open since R11:** The check is not independent of the string: the 40 papers make up 40 of the 51 included, came from exploratory searches with similar words and were known while the protocol was written. Give the interval (Wilson 87–99.6%) and the date the string was frozen. Deep research found a second miss in the same sub-area: DependaFix (FSE 2026 demo, doi 10.1145/3803437.3806404) repairs breaking dependency updates via Byam and was not a keyword hit. Run a supplementary string over the 4,021 non-hits (convert*, rewrit*, update*, port, breaking change, dependency update, API evolution) and one round of snowballing (Wohlin, EASE 2014) from the 51 included papers, and report what they add.**]** We added the one miss, a dependency-repair paper whose title and abstract use none of our terms [15], to the candidates as a known paper, which gives 431 candidates.

4.3 Selection

A paper is included if it satisfies I4 and at least one of I1–I3, provided that no exclusion criterion applies (Table 3). The criteria follow from the RQs. I1 admits papers that use a neural model or an LLM to migrate code and say how equivalence is checked, even if only by BLEU, so that every included paper has an oracle to code.**[P34, Minor; R2; open since R12:** The title says LLM-based, but I1 admits any neural model: MBTA evaluates TransCoder and j2py (rule-based) with no LLM, yet its LLM role is E (it should be N), and the BatFix row of Figure 3 and the UniTrans and TransAgent-baseline rows of Table 9 show TransCoder outputs. Define LLM inclusively in Section 2 or adjust the title, and report how many papers evaluate only pre-LLM models.**]** I2 adds papers on benchmarks and evaluation methods, since these define oracles even when no translator is proposed. I3 covers papers on detecting whether two programs in different languages are equivalent. Their detectors could serve as oracles, but the papers evaluate the detectors rather than migrations, so they form a separate tier, which the results report apart from the main corpus (Section 5.5). Secondary studies (E4) belong in related work (Section 3), not in the map.

Hard cases were discussed within the team until agreement was reached**[P35, Major; R1, EIC; open since R13, R30:** The most likely reason for a major revision at EASE. The paper never says how many people screened the 431 candidates, read the 66 full texts or coded the 49 papers, whether any step was independent, how agreement was measured, or whether papers coded before the scheme was refined were recoded. The 30/10 version deleted the calibration paragraph without replacing it. Yet G2 and checklist item 3 fault primary studies because only 3 of 17 report rater agreement. Minimum: state who did each step and report agreement on a calibration sample (e.g. a second author, blind, recodes 10–15 papers on fields 4, 5 and 7 and re-screens a random sample of title/abstract exclusions; Cohen's kappa per field, raw agreement or PABAK for rare codes). If that is impossible, say that one person coded and list it under Reliability.**]**, and nine clarification rules record the outcome (Table 4). Each rule settles how the criteria apply to papers that their wording leaves open.**[P36, Minor; R1, R2:** State when each rule was introduced and confirm that all 431 candidates were

Table 4: Clarification rules: how the criteria in Table 3 apply to hard cases. N: number of the 431 candidates that the rule decided.

Rule	Decision	N
L-TEST	Tests count as code: migrating tests with a neural model or LLM meets I1	1
L-BASE	A study proposing a deterministic technique meets I1 if the technique and an LLM baseline are evaluated with the same oracle and both results are reported. This requirement aligns paper-level oracle coding with the evaluation of LLM-generated outputs.	2
L-ARCH	Microservice extraction and other architecture changes do not meet I1: they restructure a system rather than move its code to another language, version, library, dialect or framework	6
L-DECOMP	Decompilation does not meet I1: it starts from a binary rather than from source code	5
L-DATA	A dataset without an evaluation protocol is E2; one with a protocol meets I2	0
L-RB	An oracle or benchmark for rule-based translators still meets I2	1
L-XEQ	Cross-language equivalence detection meets I3, even when a paper frames it as clone detection	2
L-TASK	Translation must be a main goal or research question to meet I1 or I2; general models or benchmarks in which translation is one of three or more tasks, and papers that use translation only as a means, are excluded	16
L-SUM	A paper that summarizes its authors' published results counts as a duplicate (E5)	3
Total		36

re-screened under the final rule set (four rules decide only one or two papers each). Justify L-TASK's three-or-more-tasks cut: deep research suggests it excluded Galapagos on its abstract (see Section 5.3).] The rules decided 36 of the 431 candidates (31 exclusions and 5 inclusions), and the replication package lists the rule for each. In total, 51 candidates were included and 380 excluded. The reasons for exclusion were: neither translation nor its evaluation (174)[**P37, Minor; R1; open since R14:** 270 of the 380 exclusions use two catch-all reasons that Table 3 does not define, plus same-language equivalence detection (15); no precedence says which reason is recorded when several apply, and rule-based exclusions are counted unevenly (L-TASK has its own line, L-ARCH, L-DECOMP and L-BASE sit inside the 96, L-SUM inside E5). Define or merge these reasons in Table 3.]; related topics that meet none of I1–I3, such as purely rule-based translation, refactoring with no target or program repair (96); natural-language tasks (E1, 47); translation as a minor task (L-TASK, 16); same-language equivalence detection (15); no evaluation information (E2, 14); clone detection unrelated to equivalence (E3, 8); duplicates (E5, 7); and no available full text (E6, 3).

Figure 1 shows the flow. We excluded 362 of the 431 candidates on title and abstract, 3 of them on the title alone because no abstract was available; Section 4.4 reports the full-text stage.

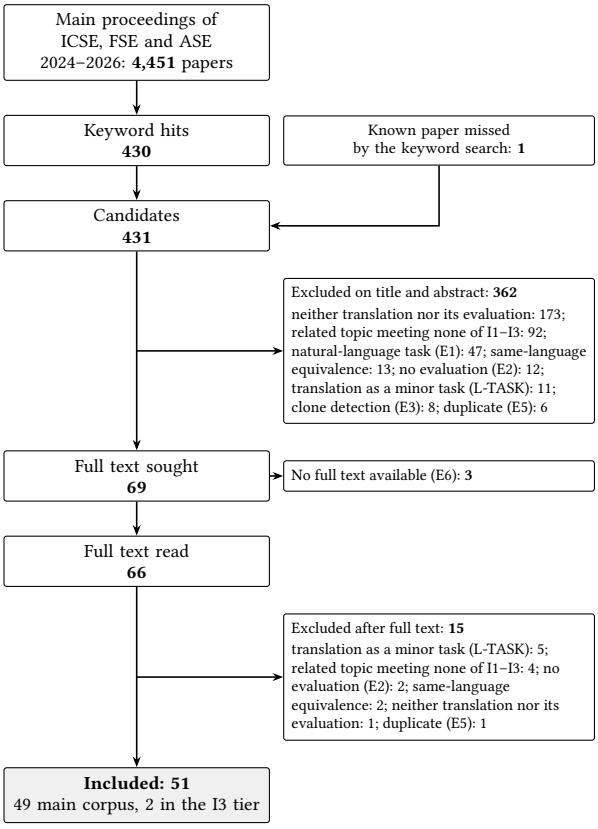


Figure 1: [new] Selection flow. Counts are papers; reasons follow Tables 3 and 4.

4.4 Full-text confirmation

No paper was included on title and abstract alone. We sought the full text of 69 candidates: those that looked eligible and those that could not be decided without it. Three were available only behind an IEEE paywall and were excluded under E6;[**P38, Major; R2, R1, DR; open since R15:** Now easy to fix. DLBench (ASE 2025 research track, an LLM benchmark for SQL dialect translation) has a free author copy at matafeiyanll.github.io/paper/ASE25.pdf; it argues that execution-only checking misses semantic equivalence and dependency integrity and would be I2, DIA (a kind with only 2 papers). Request Polyglot (ASE 2025) from the authors or the library; Rubhus (ICSE 2024 JF) is in an open PhD thesis (I3 tier at most). PRISMA 2020 reports unobtainable papers as not retrieved, not as an exclusion criterion.] the other 66 came from freely available versions (arXiv, an institutional repository, PubMed Central) or the ACM Digital Library. Of these 66, 51 were included and 15 excluded; Figure 1 gives the reasons. The full text decided several borderline cases.[**P39, Minor; DR, R2:** Deep research found three decisions to recheck. (1) Migrating Code At Scale With LLMs At Google (FSE 2025 Industry, C185) was excluded as having no named source-target pair, but it reports a named int32-to-int64 ID migration checked by build, regression tests and human review, which the included googlemig also covers: include it, or add a rule for type migrations and correct the stated reason. (2) C317 (ICSE 2024

poster) was excluded under E5 in favour of a later MSR 2024 version that lies outside the scope, so the work leaves the map. (3) C320's E2 reason (problem statement) does not match E2's definition.] One translation paper states that it does not check whether its output is correct (E2). Two others meet none of I1–I3: one only adds type annotations to JavaScript code, and the other verifies whether a PLC program preserves its existing functionality after feature additions. Neither study concerns migration between the source and target technologies covered by our scope. A deterministic converter between deep learning frameworks fails L-BASE because its LLM baselines are scored by a different oracle; a poster by the same group, which converts between the same frameworks with LLMs, meets I1 and is included [31].

4.5 Classification and extraction

We code each paper with the two classification schemes of Section 2. The migration kinds correspond one to one to the source and target types that criterion I1 lists (Table 3); every included paper has at least one. The nine oracle categories began as a scheme that we drafted during protocol development from the papers we knew before the search, and we refined it during extraction. For example, formal verification was separated from property-based testing, and the test-based categories were redefined by the origin of their expected outputs. For each included paper we drafted values for the seven fields of Table 5 from the full text and recorded short quotations, with their locations, as evidence. Field 5 splits the four test-based categories into the subtypes listed in the table; for differential testing it records the source of the inputs, because the expected outputs are always those of the original.

We then applied the following rules to turn the drafts into final codes. Only oracles that judge the final output count; one used solely inside the technique, such as a compiler in a repair loop or a filter on training data, does not. Seven further rules handle boundary cases. The paper must report the oracle's result on the evaluated outputs. Compilation or a syntax check counts as O-b whenever the paper reports how many outputs pass it, even if only as a breakdown of failures by stage or in a figure; compilation implied only by passing tests is not coded.**[P40, Minor; R2, R1:** Two identical evaluations can be coded differently (one prints a failure breakdown, the other does not), and a compile count in a breakdown does not decide whether the migrated program behaves like the original (your definition of an oracle). Report compile rates used as a success metric separately, and consider a strongest-oracle-per-paper view (no executing oracle / tests / differential / formal) that answers RQ1 directly.] If a system translates tests along with the code, checks that touch only the translated tests, such as compiling them, are not oracles of the translation. In test migration, where the migrated test is itself the output, running it on the target counts as O-d. Tests translated by hand when a benchmark was built are benchmark tests (O-c), whereas tests that a paper's authors translate for their own evaluation are translated tests (O-d).**[P41, Minor; R2:** The O-c/O-d boundary follows timing, not origin: hand-translated tests are O-c if a benchmark builder translated them and O-d if the study's authors did, and HumanEval/131 shows that O-c tests share O-d's failure mode. O-d also merges tests translated by the system under evaluation (not independent) with tests translated by hand (independent). Add independence (pre-existing, built by the authors, produced by the system) as an explicit axis; G4 needs exactly that.] LLM-written tests whose expected outputs have no stated source are coded O-e, since nothing shows that they were checked against the original. The role of the LLM follows what the LLM does in the study, whatever kind of technique is proposed; a non-LLM technique compared with LLM baselines is coded as baseline only. We also re-checked every number in the disagreement analysis (Section 5.4) against the paper.

The analysis counts papers. Section 5.1 describes the corpus with fields 1 to 3, and RQ1 counts the oracles of field 4 (Section 5.2). RQ2 reports the origins of field 5 and compares oracle use across four contexts: a legacy source or not, the migration kind, the track and the publication year (Section 5.3). RQ3 draws on fields 6 and 7. For the papers with human evaluation, we also record whether they report agreement between raters (Section 5.4). Appendix A gives the codes for every paper. We did not score the quality of the included papers: the RQs ask how they evaluate equivalence, and RQ3 covers the weaknesses that they state or show.

Table 5: [new] Extraction form.

Field	Values
1. Migration kind	XL, VER, LIB, DIA, FW (several allowed); legacy flag
2. Track	Research, industry/SEIP, journal-first, NIER, Ideas, Visions and Reflections, poster, demonstration
3. Role of the LLM	Proposed technique; evaluated subject; both; baseline only; part of the evaluation tooling; no LLM (a smaller neural model or no model)
4. Oracles	O-a to O-i (Table 1); final-output oracles only
5. Origin of test-based oracles	O-c: benchmark or project tests; O-d: translated by the system or by hand; O-e: expected outputs from a specification or not stated; O-f: where its inputs come from (fuzzing, property-based testing or symbolic execution; existing tests or data; an LLM; curated samples; not described)
6. Oracles on the same outputs	Yes or no; if yes, the numbers under each oracle
7. Stated limitations	Free text with location, later grouped into themes with one main theme per paper**[P42, Major; R1; open since R16:** How is the main theme chosen when a paper states several (first mentioned, most space, most severe)? RQ3 asks which threats studies acknowledge, so one label per paper undercounts every secondary theme, yet the text reads theme counts as mention counts (named by half of the 28 papers; 3 of these 14 name subjective evaluation). Code several themes per paper (the free text exists) and report any-mention counts next to the main-theme counts. Field 2 also lacks the doctoral symposium and SRC tracks that Section 4.1 puts in scope.] (Table 10)

5 Results

Counts below refer to the 49 papers of the main corpus; the two cross-language equivalence-detection papers (I3) are reported in Section 5.5. A paper can use several oracles and involve several migration kinds, so the counts in a figure or table can add up to more than 49.

5.1 Corpus overview

The corpus grows quickly: 8 papers are from 2024, 14 from 2025 and 27 from 2026, or 0.68%, 0.98% and 1.45% of the papers scanned in each year (Table 2). The 2024 share may be slightly low, since 61 ICSE 2024 papers had no abstract and were matched on their titles only.

Most papers come from the research track (28); the others come from industry/SEIP (9), journal-first (5), NIER (3), posters (2), demonstrations (1) and Ideas, Visions and Reflections (IVR, 1). We call the NIER, IVR, demonstration and poster papers short papers. Translation between languages dominates (XL, 33 papers), followed by library migration (LIB, 12), version migration (VER, 3), framework migration (FW, 3) and dialect translation (DIA, 2); 5 papers migrate legacy systems, as defined in Section 2. Rust is a target language in 13 papers.**[P43, Minor; open since R17; DR:** This cannot be checked from the paper (the Appendix has no target-language column); add one or drop the number. Expand NIER and SEIP at first use, and PLC in Section 4.4. Deep research: scanned_papers.csv labels 64 FSE 2024 papers as posters although many are research papers (e.g. CodePlan, correctly Res. in the Appendix); recheck any per-track denominators.**]** The LLM is the proposed technique in 32 papers, the evaluated subject in 9 and both in 4; in the remaining 4 it is only a baseline (3, 2 of which propose a smaller neural model) or part of the evaluation tooling (1).

5.2 RQ1: Oracles in use

Compilation and existing tests dominate; formal verification, LLM-generated tests and LLM judges are rare (Figure 2). Every migration has the original program as a reference, yet only 14 papers (29%) execute it to compare behavior, all by differential testing. Because an oracle is counted whenever a paper reports its result (Section 4.5), the counts record which oracles the papers use, not which ones their conclusions rest on.

Most papers (40 of 49) combine oracles: 16 use two, 18 use three and 6 use four or more, up to five in RustAssure [3]. Nine papers use no oracle that executes the code (none of O-c to O-f) and judge outputs by compilation, similarity, a human or an LLM. Five of them rely on a single oracle: BLEU or CodeBLEU in two [12, 30], an LLM judge in one [7] and human judgment in two [29, 52].

Table 6 shows which oracles are combined. Compilation accompanies most other oracles: 23 of the 28 papers with existing tests and 12 of the 17 with human evaluation also use it. The remaining 5 of the 28 report no compile rate, so for compiled targets a translation that fails to compile cannot be told apart from one that compiles but fails the tests. Of the 14 papers that use differential testing, only 4 combine it with existing tests; in 9 it is the only test-based oracle, and the last pairs it with translated tests. Both papers with formal verification also use differential testing, in VERT [58] as a step before model checking and in RustAssure [3] as part of the same symbolic check.

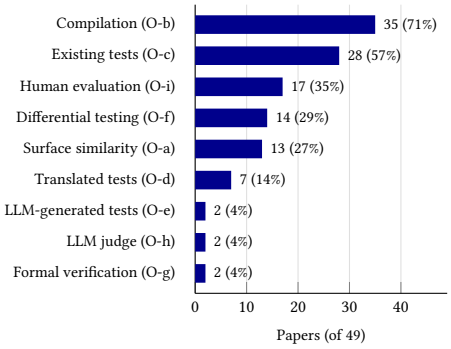


Figure 2: [revised] Number of main-corpus papers (of 49) that report a result for each oracle on the final outputs. A paper can use several oracles, so the bars sum to more than 49.

Table 6: [new] [revised] Oracle combinations in the main corpus. Each cell counts the papers that use both oracles; the diagonal gives the papers that use each oracle.

	O-a	O-b	O-c	O-d	O-e	O-f	O-g	O-h	O-i
O-a	13								
O-b	7	35							
O-c	7	23	28						
O-d	0	7	5	7					
O-e	1	1	1	0	2				
O-f	1	12	4	3	0	14			
O-g	0	2	0	1	0	2	2		
O-h	0	1	0	0	0	0	0	2	
O-i	4	12	6	3	0	5	1	1	17

5.3 RQ2: Ground-truth origin and context of use

Where the ground truth comes from. Table 7 breaks down the four test-based oracles by the origin of their ground truth; we do not code the origin of the other oracles. Existing tests split almost evenly by who wrote them: benchmark builders in 14 papers, a real project's developers in 12 and both in 2 [14, 26]. Among the 14 LIB and VER papers, 7 use existing tests, and 6 of these 7 run the project's own suite on the migrated code [15, 27, 37, 40, 48, 55]. In translation between languages, benchmark tests are more common: 12 papers, against 6 with project suites and 2 with both.

The system under evaluation produces the translated tests in 5 of the 7 papers that use them. In 4 of these the same system also translates the code, and a translation error can therefore affect the code and its tests alike; in IntentTester, which migrates only tests, the migrated tests are themselves the output under evaluation, so a passing test shows only that its assertions hold on the target library, not that they still check what the original test checked [18]. The authors translate tests by hand for their own evaluation in 3 of the 7 papers [3, 26, 43]. Two papers use LLM-generated tests in the sense of Table 1, where the expected outputs come from an LLM or a specification: one derives its tests from the same specification it uses to generate the code [6], and one does not state the origin of

Table 7: [new] [revised] Origin of the ground truth of test-based oracles in the main corpus; for differential testing, origin of the inputs. Per-paper codes are in Appendix A. One paper [26] appears in two O-d rows and in two O-f rows, so the O-d rows sum to 8 and the O-f rows to 15.

Oracle and origin	Papers
O-c Existing tests (28)	
Built for a benchmark (curated problems, reference translations)	14
A real project's own test suite	12
Both	2
O-d Translated tests (7)	
Translated by the system under evaluation	5
Translated by hand by the authors	3
O-e LLM-generated tests (2)	
Derived from the specification that also generates the code	1
Origin of expected outputs not stated	1
O-f Differential testing (14)	
Inputs from fuzzing, property-based testing or symbolic execution	5
Inputs from existing tests or benchmark data	3
Inputs generated by an LLM	3
Curated samples or rendered screens	2
Inputs not described	2

its expected outputs [60], so by the rule in Section 4.5 we code it as O-e.

Differential testing always compares outputs with the original program, so for this oracle we code the origin of its inputs: fuzzing, property-based testing or symbolic execution in 5 papers, existing tests or benchmark data in 3 and an LLM in 3 [2, 26, 33]. Java-to-ArkTS runs both versions on curated samples [32], GUIMigrator compares the rendered screens before and after migration [17] and 2 papers do not describe their inputs; ReCodeAgent has two input origins [26].

How oracle use differs by context. Table 8 splits the corpus by legacy status, migration kind, track and year. The legacy, VER, FW, DIA and journal-first groups have five papers or fewer each, so patterns in them are illustrative only.

- **No legacy paper uses existing tests.** Two of the 5 legacy papers run the original system as a differential oracle [16, 23], one generates tests with an LLM from an extracted specification [6], one uses an LLM judge [7] and one compares the generated Java with the expected code by hand [52]. This is the one empty cell that chance does not easily explain: with 28 of the 49 papers using existing tests, about 3 legacy papers would be expected to use them.**[P44, Minor; R1, DA, R2; open since R18:** This is an inferential claim, while Section 7 says you do not test differences. The cell was singled out after looking at 126 cells (P = 0.011 for 0 of 5, hypergeometric, but about 0.08 across all cells), and LIB with human evaluation departs more (P = 0.0014) without such a comment. The simplest explanations are missing: legacy systems often have no automated tests, and the cell is produced by a definition that excludes C. Drop the phrase or give the post hoc probability with these caveats.**]** With 5 papers, all in the industry/SEIP or NIER tracks, legacy cannot be separated from track. The result also depends on our definition: the legacy flag does not cover C, yet at least 4 C-to-Rust papers call their C sources legacy and use existing tests [14, 33, 50, 53].
- **Library migration often includes a human check.** Nine of the 12 LIB papers include human evaluation, against 6 of 33 XL papers. Six of these 9 also execute the code, and the other 3 judge behavior without executing it [1, 29, 35]. Several human checks cover only part of the outputs, such as a 20% sample of the patches that pass the tests [15] or the 24 cases that have a developer's fix [55].
- **Both formal-verification papers target Rust** [3, 58]; with two papers, this is an observation rather than a pattern.**[P45, Minor; R2, DR:** Deep research: Galapagos (FSE 2026 journal-first, candidate C232, excluded on title and abstract under L-TASK) compares a test suite with formal equivalence checking (alive-tv on LLVM IR) on LLM-generated C-to-Go variants: 207 of 300 pass the tests, only 73 are proven equivalent. Re-screen it on the full text; it is a non-Rust formal-verification case and a strong same-output disagreement example.**]**
- **All 5 journal-first papers use existing tests**, and none uses human evaluation.
- **LLM-based oracles appear only in 2026.** The 2 papers with LLM-generated tests [6, 60] and the 2 with LLM judges [7, 35] are all from 2026. Earlier papers use LLM-generated tests only inside the technique [30, 59], and with four papers the timing may be chance.
- **Other zero cells** include legacy with O-a, O-d or O-g; LIB with O-e or O-g; DIA with O-c, O-d or O-e; and short papers with O-d, O-e, O-g or O-h. If oracle use did not depend on the group, each of these cells would hold at most 1.3 papers, so an empty cell says little on its own.

5.4 RQ3: Threats to oracle reliability

Oracle disagreement. Thirty-three of the 40 papers that combine oracles (Section 5.2) report separate numbers for two or more of them on the same outputs. The other 7 do not: four apply the oracles as successive checks and report only the final outcome [6, 40, 41, 48], one gives a number for a single oracle [31], one applies them to different outputs [38] and one shows the per-oracle results only in a chart without values [34].

Of the 33, 18 give both a compile rate (or the pass rate of another well-formedness check) and a behavioral rate (Figure 3), and 6 more compare other oracles (Table 9). The remaining 9 fit neither the figure nor the table: they count compile errors or bugs rather than outputs [2, 50], pool failures over several models [42, 43], measure compilation and equivalence in different units [3] or report similarity scores or manual judgments with no behavioral pass rate on the same outputs [1, 4, 17, 27].

Figure 3 and Table 9 show the limits of compilation, surface similarity and testing. Compilation accepts many outputs that fail the behavioral check: the behavioral rate trails the compile rate by 3 to 98 percentage points across the 18 configurations in Figure 3, and by more than 10 points in 13 of them.**[P46, Major; R1, DA, R2, EIC; open since R21:** No rule says which configuration represents

Table 8: [revised] Oracle use by context. Each cell counts papers in the group that use the oracle; *n* is the group size. Per-paper codes are in Appendix A. A paper with two migration kinds counts in both rows, so the migration-kind rows sum to 53. IVR: Ideas, Visions and Reflections.

Group	*n*	O-a	O-b	O-c	O-d	O-e	O-f	O-g	O-h	O-i
Legacy	5	0	1	0	0	1	2	0	1	1
Not legacy	44	13	34	28	7	1	12	2	1	16
XL	33	7	23	20	6	2	11	2	1	6
LIB	12	4	9	6	2	0	1	0	1	9
VER	3	1	2	1	0	0	1	0	0	1
FW	3	1	3	1	0	1	1	0	0	1
DIA	2	1	1	0	0	0	1	0	0	2
Research	28	8	19	15	6	1	8	2	1	13
Industry/SEIP	9	2	7	5	1	1	3	0	1	2
Short: NIER, IVR, demo, poster	7	2	4	3	0	0	2	0	0	2
Journal-first	5	1	5	5	0	0	1	0	0	0
2024	8	3	5	5	1	0	2	0	0	1
2025	14	3	10	8	3	0	4	2	0	6
2026	27	7	20	15	3	2	8	0	2	10

each paper, and 4 of the 18 rows are not the papers' own systems (two baselines, an ablation, pre-repair output); RISE is coded P but shown with a GPT baseline. The 98 is the compile-only-reward ablation, the 10-point cut behind 13 of 18 is not justified, and several rows sit near it (TerraMod 8, RISE 8, ReCodeAgent 13). The caption says a gap is meaningful only within its row, yet the text aggregates across rows. State the rule in advance (e.g. the paper's headline LLM configuration on its main benchmark), report the median and IQR (median now 19 points), and give the range with and without the ablation.] Similarity scores do not reliably reflect test results. Exact match rejects most translations that pass the tests in both papers that report the two [13, 59]. In 4 of the 5 papers that report a similarity score next to a pass rate on the same outputs, the score ranks at least one pair of systems or settings in the opposite order to the pass rate [13, 14, 22, 59].**[P47, Minor; DA; open since R22:** At least one pair is lenient: in the pairs shown for CAST, UniTrans and DepTrans the similarity scores differ by about 1–1.5 points, and only TerraMod reverses strongly. Report, per paper, the pairs compared and reversed with their sizes, or a rank correlation.] For TerraMod and DepTrans, a variant that passes fewer tests even scores higher than the proposed system [14, 22]. Tests and fuzzing also accept outputs that a further check rejects. All 4 papers that apply such a check find these outputs in every setting they report [10, 15, 55, 58]. How many fail it differs greatly: one of the 24 BDUpdater updates, against 16 of the 23 Code hints candidates without code hints.**[P48, Major; R2; open since R24:** Per R2 (checked against arXiv 2412.08041v4, Table 1), the Code hints numbers here and in Table 9 come from the paper's non-LLM symbolic engine (RefSym). For its LLM engine (Claude 2.1 or 3), manual review rejects only 1–2 candidates per setting (e.g. 14 vs 12 without hints), so differs greatly no longer holds for LLM outputs. Use the LLM numbers and note that both engines search against the same fuzzing check (a G4 case). Please verify against the paper.] In MBTA, TransCoder reaches perfect CA on 168 programs yet mistranslates at least one mutant of each [21].**[P49, Minor; R2; open since R23:** MTS is computed on translations of mutants, i.e. other outputs, so counting MBTA among the 33 same-output papers conflicts with excluding MSA for applying oracles to different outputs. The sentence also reads as if CA = 1 on exactly 168 programs (CA = 1 on 29.62% of the programs, 168 of which have a mistranslated mutant). Discuss MBTA separately as evidence on test adequacy and translator robustness; 33 becomes 32 unless its compile rate vs CA qualifies.] A translation that passes all its tests therefore does not show that the translator handles small changes to the program. The rows of Table 9 use different measures and cannot be pooled.

Limitations that studies report. Weak or few tests are the most frequent main limitation (Table 10), named by half of the 28 papers that use existing tests. Of the 5 papers that rely on a single oracle without executing the code (Section 5.2), 4 state no limitation of that oracle [7, 12, 30, 52]. The themes record what papers state, not what their authors know. Each paper on oracle gaming shows a different form. A DepTrans model rewarded only for compiling emits trivial code such as `panic!` and passes no tests (Figure 3) [14]. SmartC2Rust observes its LLM "simplifying and rewriting the code just to make the test cases pass" on a large program outside its main evaluation [50]. In FreshBrew, agents exclude failing tests from the build or make them skip themselves, and a rule that rejects any migration losing more than 5 percentage points of test coverage catches only a clear subset of such attempts [37]. IP-preserving illustrates the same-source theme: tests and code are generated from one specification, so passing the tests shows consistency with that specification rather than with the original system [6].

Agreement between human raters. Of the 17 papers with human evaluation, 3 report agreement between raters [15, 27, 57]. The other 14 give no such measure, which leaves the consistency of their judgments unknown. Only 3 of these 14 name subjective evaluation or the missing measure as their main limitation (Table 10) [18, 38, 44].

Table 9: Other oracle comparisons on the same outputs. Each group includes all main-corpus papers of its kind. Outputs judged: one or two of several settings per paper. Results follow the order of the oracles compared, with scores on each paper's own scale. Four papers also appear in Figure 3. CA: computational accuracy, the share of outputs that pass the tests. MTS: mutation-based translation score, the share of a program's mutants that the translator gets wrong.

Paper	Outputs judged	Oracles compared	Result
Similarity next to a pass rate			
UniTrans [59]	LLaMA-33B, 1-shot, average of four translation directions	Tests (CA) vs exact match	47.63% vs 9.16%
	TransCoder-ST, average of four translation directions		52.15% vs 7.70%
CAST [13]	Qwen2.5-72B, 1-shot, average of four translation directions	Tests (CA) vs exact match	68.36% vs 15.67%
	LLaMA3-70B, 1-shot, average of four translation directions		65.67% vs 16.76%
TransAgent [60]	TransCoder (a baseline), Python to Java	Tests (CA) vs CodeBLEU	2.4% vs 40.1
	TransAgent, Python to Java		89.5% vs 69.8
TerraMod [22]	LLaMA-4-Maverick without migration context (Base)	Execution (Pass@1) vs CrystalBLEU	44% vs 0.78
	LLaMA-4-Maverick with migration context (TerraMod)		76% vs 0.66
DepTrans [14]	RAST-7B, compile and test reward weights 1:1, both benchmarks	Tests (CA) vs CodeBLEU	43.5% vs 65.4
	RAST-7B, compile and test reward weights 1:2 (ablation), both benchmarks		40.8% vs 66.5
Further check on outputs that pass tests or fuzzing			
BDUpdater [55]	GPT-4.1, 24 updates	Existing tests vs developer's fix	24 vs 23
Code hints [10]	Symbolic engine, deprecated methods with code hints	Fuzzing check vs manual review	112 vs 104
	Symbolic engine, deprecated methods without code hints		23 vs 7
Fruntke & Krinke [15]	Zero-shot, 20% sample of the patches that pass the tests**[P50, Minor; R2; open since R24:** Figure 3 plots zero-shot Claude 3.5 Sonnet with 12 of 65 passing patches, so a 20% sample would be 2–3 patches, not 10. Say which models or runs the sample pools and what 9 and 8 count (correct patches per engineer? on which criterion?).**]**	Existing tests vs manual grading by two engineers	10 vs 9 and 8
	Agentic, 20% sample of the patches that pass the tests		16 vs 13 and 11
VERT [58]	Claude-2 + VERT, 569 C++ programs	Compilation vs property-based tests vs bounded verification vs full verification	539 vs 295 vs 233 vs 19
Mutation score next to CA			
MBTA [21]	TransCoder, each of 168 programs	Tests (CA) vs MTS	CA = 1 vs MTS > 0

5.5 Cross-language equivalence detection (I3)

Two further papers concern tools that decide whether programs in two languages are equivalent. Neither evaluates a migration, so both are outside the 49 papers counted above. We report them because they examine two kinds of oracle that main-corpus papers use: an LLM judge (O-h) and formal verification (O-g). Moumoula et al. ask five LLMs whether two programs in different languages are clones and score the answers against known labels. The best of the five, GPT-3.5-Turbo, reaches an F1 score of 0.99 on XLCoST but 0.82 on CodeNet [39]. All five score lower on CodeNet, and the authors attribute the drop to its more complex problems. The accuracy of an LLM judge is therefore not fixed: a figure measured on simple programs need not hold for harder ones. Both main-corpus papers with an LLM judge omit such a figure for their own judge [7, 35]. Without an LLM, EquivFusion lowers a C, C++ or PyTorch specification and a Chisel, Verilog or gate-level design into MLIR and checks their equivalence with SAT and SMT solvers [64]. Its two case studies are small, sorting 8 numbers and a dot product of 2-element vectors. In each, the solvers prove one design pair equivalent and find a mismatch in another. Formal checking across languages is thus not limited to Rust, the target of both main-corpus papers with formal verification (Section 5.3), but the paper does not show that the tool scales to larger designs.

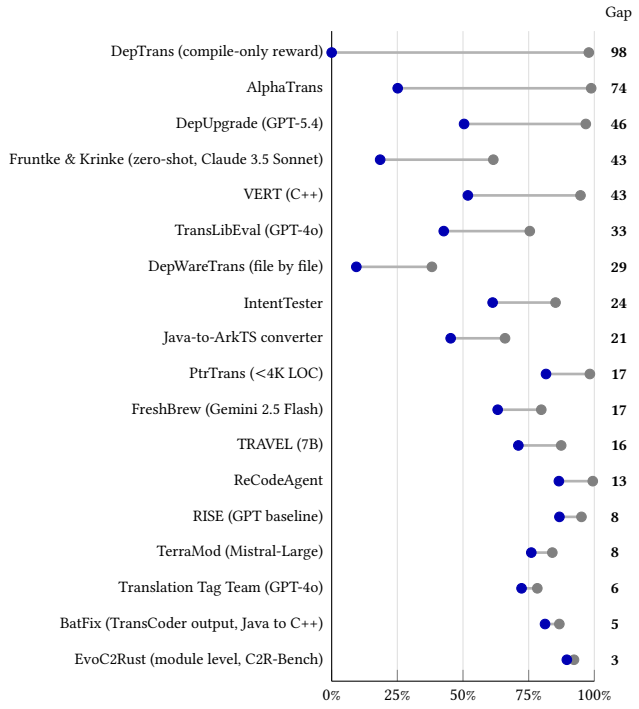


Figure 3: Share of outputs that pass compilation or another well-formedness check (gray) and share that pass the behavioral check (blue) in the 18 papers that report both for the same outputs. Numbers on the right give the gap in percentage points. Each row shows the configuration in parentheses or otherwise the paper's own system; the RISE and DepWareTrans rows are baselines, the DepTrans row is an ablation and the BatFix row shows the translations before repair. Rates are as reported or computed from reported counts, except that DepUpgrade and IntentTester report the behavioral rate over the outputs that compile and we recompute it over all outputs. The behavioral check differs by paper (DepUpgrade uses human review, IntentTester's outputs are tests and ReCodeAgent counts passing tests rather than programs), so a gap is meaningful only within its row. The replication package lists each configuration and where the paper reports it.

6 Pilot: One Set of Translations, Four Oracles

RQ4 asks how automatic oracles that do not execute the code compare with existing tests on the same LLM translations. We hold the translations fixed and vary only the oracle: compilation (O-b), existing tests (O-c), surface similarity (O-a) and an LLM judge (O-h). The first three are common in the map (35, 28 and 13 of the 49 papers); LLM judges are new, appearing in 2 papers from 2026, neither of which checks its judge against tests and one of which relies on it alone [7, 35]. Of the four, only existing tests execute the translation, so we use them as the reference, although passing them does not prove equivalence for every input.**[P51, Major; R1, DA, R2:** The reference is a suite that your own mutation check shows to be weak (16% of compiling mutants survive), and no disagreement was adjudicated, yet Table 11's False accept and False reject and the text (errs, only the tests reject it) treat the test verdict as truth. The oracles also use different references (Java reference for CodeBLEU, Python code for the judge, Java tests for O-c). The Python originals run, so add the oracle the paper recommends most (G5): run original and translation on a larger input set (e.g. EvalPlus/HumanEval+ inputs converted to Java) and use passes the tests and matches the original as the reference; adjudicate the 76 disagreement tasks or a sample with two raters. Until then rename the columns (e.g. accepts test failures).]

Table 10: Oracle limitations that papers state about their own evaluation. Each of the 49 main-corpus papers is counted once, under its main theme.

Theme	Papers
Weak or few tests, so passing tests does not imply equivalence	15
Incomplete differential, fuzzing, symbolic or bounded checks	6
Subjective human evaluation, or no agreement measured	3
The LLM games the oracle: optimizes for compiling or passing tests	3
Oracle derived from the same source as the output	3
Misleading surface similarity	3
Compiling does not imply correct behavior	2
No executable oracle available	2
Other limitations of the oracle (e.g., readability)	2
No limitation of the paper's own oracle stated	10

6.1 Design

Tasks and generation. HumanEval-X [63] provides each of its 164 problems in several languages, including Python and Java, with a reference solution and tests in each. We translate every task from Python to Java, not the reverse, because only a compiled, statically typed target makes compilation a meaningful check. The translator receives the Python implementation and the benchmark's Java class and method signature, but neither the Java reference solution nor the tests, and produces one translation per task, without a test-guided repair loop. We remove the docstring from the Python code so that the model translates the code rather than solving the task from its description. The translator is Qwen2.5-Coder 7B Instruct [24] and the judge Llama 3.1 8B Instruct [19], both quantized to 4 bits (q4_K_M) and served by Ollama 0.40.11 with temperature 0, seed 1234, a 4,096-token context and output limits of 1,024 tokens for translation and 128 for judgment. A judge from another model family avoids having a model grade its own output. The single run (October 25, 2026; Tesla T4 GPU; OpenJDK 21.0.12) is saved with the model digests, prompt and dataset hashes and the executed notebook.

Oracle implementation. Compilation runs `javac` on the extracted Java class, preceded by the task's imports, and times out after 60 seconds. The test oracle compiles the translation together with the benchmark tests and runs them for at most 10 seconds; any

compilation error, nonzero exit or timeout counts as a rejection. Similarity is CodeBLEU [47] (version 0.7.0)**[P52, Minor; DR, R2:** 0.7.0 is the version of the unofficial codebleu PyPI package, not of Ren et al.'s metric: write the codebleu Python package (version 0.7.0), an implementation of Ren et al.'s metric.**]** against the Java reference solution. No CodeBLEU threshold is validated for equivalence, so we report three (0.5, 0.6 and 0.7) and treat the middle one as the main threshold. The judge sees the Python implementation and the generated Java code, but not the tests or their verdicts, and is asked whether the two return the same result for every valid input. It answers YES or NO and gives a one-sentence reason; every answer could be parsed.

Checks on the harness. Before running the models, we checked the harness on answers known to be correct. All 164 of the benchmark's Java reference solutions compile, pass their tests, score CodeBLEU 1.0 against themselves and, when wrapped in an answer formatted like a model's, come out of the code-extraction step unchanged. To see how strong the tests are, we then seeded one small fault at a time into the references: each of the 298 resulting mutants changes one relational or arithmetic operator, flips one returned Boolean value or turns one assigned 0 into 1.**[P53, Minor; R1, DA:** 298 mutants over 164 tasks implies a selection from the mutation sites: state the tool or script, the operators, the sampling rule and seed, and how the one rejected mutant per task for the judge check was chosen; report survival per operator. Also classify the 46 survivors as equivalent or not and report the non-equivalent survival rate: 84% killed can equally be read as fairly strong tests, so present both readings.**]** Of the 288 mutants that compile, the tests reject 242 and the other 46 (16.0%) survive. Some survivors behave like the reference: in HumanEval/13, changing `a > b` to `a >= b` has no effect, because the method has already returned when `a == b`. Others do not. HumanEval/131 asks for the product of the odd digits of a number, or 0 if there are none. One of its mutants starts the count of odd digits at 1, so for a number whose digits are all even, such as 2468, it returns 1 instead of 0. This mutant survives because every number in the Java tests has at least one odd digit, whereas the Python tests for the same problem also check 2468. The tests therefore miss some of the seeded faults, up to 16%. This bound does not extend to faults in the translations.

6.2 RQ4: Agreement with existing tests

The three oracles that do not execute the code often depart from the test verdicts: at least one of them disagrees with the tests on 76 of the 164 tasks (46.3%). Of the 32 failing translations, 25 are accepted by one or more of these oracles but only 4 by all three. Compilation agrees with the tests most often (on 90.9% of the tasks) and errs only by accepting translations that the tests reject, whereas similarity and the judge also reject translations that pass.**[P54, Minor; R1; open since R26:** Raw agreement mostly reflects the 80.5% pass rate (an oracle that accepts everything agrees on 80.5%). Chance-corrected, Cohen's kappa against the tests is 0.65 for compilation, 0.38, 0.26 and 0.29 for CodeBLEU at 0.5, 0.6 and 0.7, and 0.26 for the judge: the judge and CodeBLEU at 0.6 are indistinguishable and CodeBLEU at 0.5 does better. Add kappa or balanced accuracy to Table 11 and reword the ordering. Also state that 76, 25 and 4 use CodeBLEU at 0.6 (64, 27, 5 at 0.5; 86, 23, 2 at 0.7).**]** A reported success rate therefore needs its oracle, its threshold and the scope of the evaluation to be interpretable; Section 8 discusses the limits of this pilot.

Table 11: The same 164 translations under four oracles, in percent. Accepts and Agrees are shares of the 164: the translations that an oracle accepts and the tasks on which it gives the same verdict as the tests. False accept is the share that the oracle accepts of the 32 translations that the tests reject; false reject is the share that it rejects of the 132 that the tests pass. Both are judged against the tests, not against true equivalence.

Oracle	Accepts	Agrees	False accept	False reject
Compilation (O-b)	89.6	90.9	46.9	0.0
Existing tests (O-c)	80.5	—	—	—
CodeBLEU (O-a) ≥ 0.5	82.3	81.1	53.1	10.6
≥ 0.6 (main)	67.1	70.7	40.6	26.5
≥ 0.7	51.8	65.2	15.6	39.4
LLM judge (O-h)	73.8	73.8	50.0	20.5

Compilation and similarity. Between 51.8% (CodeBLEU at 0.7) and 89.6% (compilation) of the same 164 translations count as correct, depending on the oracle and threshold (Table 11). Compilation accepts 15 of the 32 translations that the tests reject, so the compile rate exceeds the test pass rate by 9.1 percentage points. Of these 15, 14 fail at run time and one compiles alone but not with the tests: the benchmark's signature and tests spell the method `filterIntergers`, and the model renamed it `filterIntegers` (HumanEval/22). CodeBLEU errs in both directions: at 0.6 it accepts 13 translations that the tests reject and rejects 35 that they pass, and raising the threshold to 0.7 lowers the first number to 5 but raises the second to 52. No threshold can separate the two groups, since a translation that fails the tests scores as high as 0.98 (HumanEval/50) and one that passes as low as 0.30; translations that fail average 0.51 and those that pass 0.76.

The LLM judge. The judge accepts 16 of the 32 translations that the tests reject and rejects 27 of the 132 that they pass, so it agrees with the tests on 73.8% of the tasks, against 70.7% for CodeBLEU at 0.6.**[P55, Minor; open since R25:** Accepts and Agrees are both 73.8% for the judge, because it accepts exactly 16 of the 32 failing translations; add a footnote, or readers will suspect a copy error. See also the kappa comment above.**]** A wrong verdict can come with a plausible reason. In HumanEval/50 the translation computes `(ch - 5 - 'a') % 26`, which, unlike in Python, is negative in Java for the letters `a` to `e`. The translation compiles and scores CodeBLEU 0.98, and the judge accepts it because it "handles wrap-around cases correctly"; only the tests reject it.

The judge on known answers. Disagreeing with the tests does not by itself make the judge wrong, so we also gave it answers whose correctness is known: every reference solution and, for the 142 tasks that have one, a compiling mutant that the tests reject. It rejects 117 of the 142 mutants but also 67 of the 164 references (40.9%), which all pass the tests; its balanced accuracy, the mean of its accuracy on the references (59.1%) and on the mutants (82.4%), is 70.8%. On

25 tasks, both the reference and the model's translation pass the tests, yet the judge rejects the reference and accepts the translation. These known verdicts still rest partly on the tests: the references are benchmark solutions that pass them but are not proven equivalent to the Python code, and each mutant was chosen because the tests reject it.**[P56, Major; DA, R1:** Then the 41% headline is not yet judge error. A strict every-valid-input question can rightly reject a reference (e.g. 32-bit overflow vs Python's unbounded integers, the Background's own example). Adjudicate the 67 rejected references (or a sample, two raters, with agreement), or label them by running the Python originals and the Java references on generated inputs, and report the judge's accuracy against that label.**]**

7 Discussion

RQ5 asks what gaps separate current evaluation practice from the needs of real migration projects, and what studies should report and investigate next. A migration replaces an existing program, so it needs evidence that the migrated code behaves like the original.**[P57, Major; DA, R3, EIC, R1:** This sentence is the whole needs side of RQ5. State it as an explicit premise with a source (e.g. IEEE 1012 V&V, or the acceptance evidence that the 9 industry papers report) and say how a gap is derived from the map (which counts, what level makes a gap). Separate two purposes of evaluation: comparing techniques needs shared benchmarks and visible tests; assuring one migration needs evidence for that system; say which of G1–G5 apply to which. The map's industry papers partly cut against the framing (Google does not run the original; industry papers use differential testing about as often as research papers, 3 of 9 vs 8 of 28). Add a threat: the gaps are measured against a derived standard, not elicited from practitioners.**]** Against this need, the map shows five gaps (Section 7.1). For three of them, the pilot adds evidence from a combination of oracles that no paper in the map uses: compilation, similarity, an LLM judge and existing tests, all applied to the same translations. The gaps lead to directions for further study, implications for three audiences (Section 7.2) and a reporting checklist (Section 7.3).

7.1 RQ5: Gaps between evaluation practice and real migration

The five gaps concern three topics: oracles that do not execute the code (G1, G2); the origin and independence of tests (G3, G4); and the original program as a reference (G5).

G1. Compile rates and similarity scores diverge from behavioral results. Compilation is the most common oracle, yet neither it nor similarity checks behavior (Table 1). Nine papers use no oracle that executes the code (Section 5.2), and 3 of them describe no check other than compilation or similarity [4, 12, 30]. Where other papers report these checks next to a behavioral rate on the same outputs, they diverge from it (Section 5.4). The compile rate exceeds the behavioral rate by more than 10 percentage points in 13 of 18 papers, and similarity reverses at least one ranking of systems or settings in 4 of the 5 papers that compare it with a pass rate. On the pilot's single set of translations, compilation accepts nearly half of those that fail the tests; at each of its three thresholds, CodeBLEU both accepts some that fail and rejects some that pass (Section 6.2). Compilation still catches static errors and similarity measures closeness to a reference, but neither shows by itself that an output behaves like the original. A compile rate or a similarity score should therefore come with a behavioral rate on the same outputs.**[P58, Major; DA; open since R29:** As framed, G1 overstates. 33 of the 35 papers with compilation also execute the code, and the 18 papers behind the divergence numbers were selected because they already report a behavioral rate next to the compile rate, i.e. they already follow this recommendation. Compile rate at least behavioral rate holds by construction (nested checks), so the gap size mostly measures the translator's semantic error rate. Reframe G1 around the papers it really applies to (the 3 with only compilation or similarity; papers that report similarity without a behavioral rate) and recompute checklist item 2's baseline (CAST reports CA on the same outputs but is not among the 18).**]**

G2. Judgments by people or LLMs are rarely validated. The other 6 papers without an oracle that executes the code use human or LLM judgment [1, 7, 29, 35, 38, 52]. Such verdicts can serve as evidence of behavior, but only as far as their consistency or accuracy is known. Human evaluation is common in library migration, where 9 of the 12 papers include it (Section 5.3). Google's industrial migrations depend on it as well: LLM-generated changes go to the engineers who own the affected code, and quality "is assured through the manual review process" [40]. Yet only 3 of the 17 papers with human evaluation report agreement between raters (Section 5.4). Two of them show how, by revising the coding guideline between early batches until the reviewers agree substantially [27] or by reporting agreement for each criterion on a blind sample [15]. Neither main-corpus paper with an LLM judge (O-h) [7, 35] gives a figure for the judge's accuracy or compares it with an oracle that executes the code, although WCA4Z relies on its judge alone.**[P59, Minor; DA:** Per your own note in Section 5.5, WCA4Z states that a human study validated its judge, without figures. Say so (WCA4Z reports a human validation of its judge but gives no figures); otherwise not validated overstates. Judge accuracy: 0 of 2 in the checklist stays correct as a count of reported figures.**]** That accuracy can change with the programs being assessed (Section 5.5). The pilot measures it: on answers known in advance, a small LLM judge rejects 41% of the benchmark's own reference solutions; on the translations, it accepts half of those that fail the tests (Section 6.2).**[P60, Major; DA, R1:** Measures overstates: the judge was compared with benchmark labels that were not adjudicated, for one small model with one prompt (see the pilot comments). Write estimates its agreement with benchmark labels for one small judge.**]** Human judgments should therefore come with agreement between raters, and LLM judges with a check on known answers from the domain where they will be used.**[P61, Minor; R3, R2:** Agreement measures consistency, not accuracy: two reviewers can agree and both miss a fault. You ask LLM judges for accuracy on known answers but humans only for agreement; recommend

the same planted-defect check for human reviewers (the pilot's mutants show how) and keep agreement as a secondary measure. Cite the LLM-judge validation literature here (Wang et al., ISSTA 2025; Ahmed et al., MSR 2025 on model-model agreement; CodeJudge; ICE-Score).]

G3. Test evidence for translation rests largely on benchmarks. Tests execute the code, but in translation between languages they mostly come from benchmarks rather than from real projects. Twelve of the 20 translation papers with existing tests rely only on benchmark tests, whereas in library and version migration 6 of the 7 run the project's own suite (Section 5.3). A pass rate on benchmark problems need not carry over to code that involves persistent state, dependencies or interaction with the rest of an application. Since we did not extract test counts or coverage, the map does not measure how strong the tests are. The papers' own limitations and the pilot, however, show that benchmark tests and project suites can both miss faults. Of the 28 papers with existing tests, 14 name weak or few tests as their main limitation: 8 with benchmark tests, 5 with project suites and 1 with both (Section 5.4). In the pilot, the HumanEval-X tests miss some of the seeded faults, up to 16% of those that compile (Section 6.1). Project suites may not even reach the migrated code: in Python library migration, only 27 of the 314 migrations have a test that covers the changed code [27]. Studies should therefore state where their tests come from and how much of the migrated code the tests cover.

G4. Some tests are not independent of the system under evaluation. Evidence of behavior should not depend on the system that it assesses. Tests can depend on the system in two ways: it produces them, or it uses them to shape its outputs. In 4 of the 7 papers with translated tests, the system under evaluation translates the code as well as its tests, so one translation error can affect both (Section 5.3). The map counts papers with this setup, not how often it leads to such errors. ReCodeAgent, one of these papers, also reports results on developer tests that were translated or checked by humans and that its agents never see, to guard against errors in the tests it translates itself [26]. Three of the 49 papers state as their main limitation that the oracle and the output come from the same source (Table 10). IP-preserving, for instance, generates its tests and code from one specification [6]. The second kind arises when the system selects or repairs its outputs with the tests that later judge them. InterTrans stops its search once a candidate passes the benchmark tests on which it reports accuracy [36], and SmartC2Rust repairs its translations against the tests on which it is then evaluated [50]. Their pass rates show that the outputs satisfy the tests used to select or repair them, not that they would pass independent ones. A model that sees the tests can even game them, as when SmartC2Rust's LLM simplifies code just to make the tests pass on a large program or an agent in FreshBrew excludes failing tests from the build [37, 50]. RISE likewise checks its translations while producing them by comparing the results of the source and translated queries, the same differential test that it reports. Its evaluation, however, adds a step that the technique does not apply to its outputs: the authors confirm by hand each translation whose results match [56]. In training, TRAVEL and DepTrans reward their models with the kinds of check they report, a weaker form of this dependence, since they train on data distinct from their benchmarks [14, 33]. We noted these uses in search, repair and training as we came across them during extraction, so there may be more.**[P62, Major; R2, DA, EIC; open since R27:** For agent-based migration this is the most consequential confound, yet it rests on anecdotes. Uncounted cases include DepWareTrans (compile- and test-driven validation in its loop, reporting 100%) and Code hints (a CEGIS loop on the same fuzzing check). Add one coded field for all 49 papers: whether the reported oracle is used inside the technique and how (none, select, repair, train or reward, stopping), same suite or same kind; report it in RQ3, G4, Figure 3 and checklist item 5. Add benchmark contamination of public tests and solutions as a third kind of dependence (Riddell et al., ACL 2024; Lopez et al., TSE 2024, candidate C188).] Studies should therefore state whether the system under evaluation produced or used the tests, or was trained against checks of the same kind, and, if so, also report results on tests that it neither produced nor used.

G5. Most studies do not run the original program. The most direct evidence that migrated code behaves like the original is differential testing: running both on the same inputs and comparing their outputs.**[P63, Major; DA, R2:** A project suite that passed on the pre-migration code is already a regression oracle for the original's behavior on its inputs, whereas benchmark tests in translation come from target references or translated tests, which can drift (HumanEval/131 and /22). Separate the two groups in RQ1 and G5; the defensible gap is narrower: few studies compare outputs with the original on inputs beyond the existing tests. Also note that some migrations intend to change behavior (bug fixes, new API semantics), which a differential oracle would penalize.] Only 14 of the 49 papers use it; of the 28 papers with existing tests, 4 also run the original (Section 5.2). Running it is not always possible. K-COBAgent runs the COBOL programs it migrates on generated inputs, but only 82 of its 544 modules yield a usable oracle; the other 462 include copybooks and programs that depend on CICS, are not standalone or are interactive [16]. Hans et al. note that validation teams often lack the databases and file systems COBOL programs need, so the authors mock these resources and run the programs on a mainframe [23]. Where the original does run, the comparison covers only the inputs tried: 6 of the 14 papers name incomplete differential, fuzzing, symbolic or bounded checks as their main limitation (Table 10). For the 35 papers that do not run the original, the map does not record whether it could not be run or the authors chose not to. Studies should therefore compare their outputs with those of the original where it can be run, alongside any tests, and otherwise state what prevented it, as K-COBAgent and IP-preserving do [6, 16].

Directions for further study. Three questions for research on migration oracles follow from the gaps; for each, we note how it could be studied and what an answer would make possible.

- **Which faults does LLM-based migration introduce, and which oracles catch them? (G1–G3)** Many oracles let faulty migrations through, as G1–G3 show for compilation, similarity, LLM judges and tests. What one oracle misses, another may catch.**[P64, Minor; DA:** This rests on 4 of 32 failing translations from a 7B model, 17 of which do not even compile; the one subtle failure you analyse (HumanEval/50, modulo sign) is accepted by all three oracles that do not execute the code. Break the 25 and 4 down by kind of failure (compile error, runtime failure, wrong value) and soften: the oracles may complement each other on obvious faults and fail together on subtle ones.**]** In the pilot, the three oracles that do not execute the code seldom all accept the same failing translation (Section 6.2). Kinds of fault are widely documented: at least 18 of the 49 papers classify the failures of migrated code, from translation between languages (Lost in Translation) to library and version migration (Pig, FreshBrew) [29, 37, 43].**[P65, Minor; R1, EIC, DA; open since R28:** At least 18 papers classify failures, five papers describe faults that slip past one check, at most 3 kinds and 3 oracles per paper, and appears only in RustRepoTrans are not fields of the extraction form, and statements of absence need systematic coding. Code them for all 49 papers and add them to the recompute script, or mark them as unsystematic observations. Each direction also runs to several hundred words; for EASE cut each to its question plus one sentence on how to study it.**]** Five papers also describe faults that slip past one check but not another [10, 17, 22, 27, 37]. In Code hints, for instance, some refactorings pass a fuzzing-based equivalence check yet fail manual review [10]. These accounts cover at most 3 kinds of fault and 3 oracles per paper, so no paper systematically measures what each oracle misses. The pilot already seeds generic faults into reference solutions to assess its tests and judge (Section 6.1). A study could instead seed faults of each kind found in these classifications into target programs known to be correct and run every oracle on them, recording which kinds each oracle lets through. Researchers would then know which oracles complement each other, and teams where their own oracles leave gaps.
- **How far do published results carry over to real projects? (G3, G4)** Most translation papers with existing tests take them only from benchmarks (G3), and some use tests that the system under evaluation produced or used (G4). Results gathered this way may not carry over to real projects, either because real code differs from benchmark problems or because a system can score higher on tests that it produced or used than on independent ones. RustRepoTrans shows how much the first can matter: the pass rates of 4 LLMs fall by 16 to 31 percentage points between a benchmark of stand-alone programs and a set of functions from real projects that depend on other code in their repository [42]. DeepSeek-V3 scores 8 points below Claude-3.5 on the benchmark but 7 points above it on the real projects, so the ranking of the models changes. As for the second, ReCodeAgent's translations pass 98.1% of the tests that the tool translated itself but only 86.5% of hidden developer tests [26]. The two sets differ, however, and the authors trace some of the hidden failures to errors in how those tests were translated, so the gap need not mean that the tool's tests favor it. Among the 49 papers, the change in ranking appears only in RustRepoTrans and a comparison of pass rates on the two kinds of test only in ReCodeAgent, so neither has been examined for other systems or beyond translation. Researchers could collect real projects that their developers have already migrated, for instance to another language or to a new library, version or framework, and hold back the tests of the migrated versions for scoring, or part of them for systems using tests while migrating. Running several LLMs and published migration tools on these projects and on existing benchmarks would reveal how often the systems rank differently on the benchmarks than on the projects. Because the held-back tests pass on the developers' own migration, they would also show whether a system's pass rate on tests that it produced or used can be trusted. Teams planning a migration would then know when published results can guide their choice of system, and researchers would see which benchmarks rank systems in the same order as real projects.
- **How much is lost when the original program runs outside its full environment? (G5)** Running the original and the migrated code on the same inputs is the most direct way to check whether behavior is preserved, but the original may need an environment that a study lacks (G5). This matters especially in legacy migration, where none of the 5 papers uses existing tests (Section 5.3), so a study that does not run the original falls back on references such as a specification, an LLM judge or people (G2, G4). IP-preserving, which tests its code against a specification, names what running the original would require: a production-like instance of the legacy system, recorded traffic and data-protection rules that permit replaying it [6]. Only 2 of the 5 legacy papers run the original, and both do so without its full environment. K-COBAgent skips the programs that cannot run on their own, and most of its 544 modules are never compared with the original (G5) [16]. Hans et al. run each COBOL paragraph on inputs from symbolic execution and replace its database, file and CICS calls with mocks [23]. Both report how much of the code their comparisons reach, as the number of modules compared or as branch coverage, but checking whether these comparisons catch the same faults as running the original on recorded traffic would need the full environment that they work around. Where the full environment is still available, for instance at a company that operates the legacy system, researchers could compare the same migrated programs with the original both there on recorded traffic and outside it on generated inputs with mocked resources. Recording the faults the second comparison misses, the correct programs it rejects and the programs it cannot handle would show how much is lost and where. Tests that run the migrated code with mocks or emulators, as TerraMod does on an emulated AWS cloud [22], could likewise be compared with runs in the full environment. Teams migrating systems that are hard to run would then know whether checking outside the full environment is enough or setting it up is worth the effort, and researchers would see which kinds of program cannot yet be handled with mocks, emulators or generated inputs.

7.2 Implications

For researchers. Each of the five gaps (G1–G5) ends with a recommendation for studies that evaluate migration; the checklist in

Section 7.3 gathers these recommendations into a list that authors can follow and reviewers can check. Questions that better reporting cannot answer, such as which oracles complement each other, are taken up in the directions for further study (Section 7.1).

For benchmark builders. Many studies take their tests from benchmarks (G3), so builders can make these tests stronger and harder to game. When porting tests to another language, they could keep every case: in the pilot, one seeded fault survives because the Java tests of its HumanEval-X problem lack a case that the Python tests check (Section 6.1). A mutation check like the pilot's would flag such weak tests before release. The mutants that the tests reject and the reference solutions would also provide known incorrect and correct answers on which to check LLM judges (G2). Builders could keep part of the tests hidden from the systems they evaluate (G4). Visible tests can be gamed either by a system that selects or repairs its outputs against them, as InterTrans does in its search [36], or by an agent that excludes failing tests from the build or makes a test skip itself, as some agents do in FreshBrew [37]. FreshBrew's own coverage check catches only part of this second kind.

For practitioners migrating real systems. A team should check which oracle and which tests a published result rests on before using it to choose a tool, since oracles that do not execute the code need other evidence to show whether outputs behave like the original (G1, G2) and pass rates on benchmark problems may differ on a real project (G3). With a shortlist in hand, the team can find a suitable tool by running each one on a small part of the system to be migrated and scoring the results with tests the tools never see (G3, G4).**[P66, Major; R3:** In legacy modernization teams usually have no such tests (none of the 5 legacy papers uses existing tests). Name what teams use instead, all of them O-f variants: characterization (golden-master) tests recorded on the original before migrating, record and replay of production inputs, parallel run before cutover, shadow traffic, and canary release with rollback; and their constraints: isolating side effects, normalizing outputs (timestamps, ordering, floating point, EBCDIC vs ASCII), data protection, the cost of running two systems. 3–5 sentences, with the space taken from Directions.**]** An agent's changes to the tests or the build during the migration should be inspected, since agents in FreshBrew used such changes to sidestep failing tests [37]. Where the team relies on code review, as Google's migrations do [40], the agreement between reviewers who judge the same sample of outputs can show how reliable the review is (G2). Comparing the migrated code with the original gives the most direct evidence (G5) but may need a production-like environment, recorded traffic and permission to replay it; these are easier to secure before the old system is retired.

7.3 A checklist for oracle provenance

A reported success rate can be interpreted only if readers know how it was defined, what the outputs were compared against and how far the oracle has been checked. Table 12 lists what studies should report to give readers this information. The recommendations that end G1–G5 become items 2 to 6, and items 1 and 7 add two points outside the gaps. Item 1 asks for three details of every rate: the oracle behind it, the behavior that oracle observes and the denominator. Two success rates reported without these details may measure different things, since "equivalent" always means equivalent with respect to some oracle and some notion of observable behavior (Section 2). Item 7 asks for the limitations of the oracle, which about a fifth of the papers do not state (Table 10).

The last column gives a baseline from the map.**[P68, Minor; R3, DA, EIC:** (a) Item 4's coverage names neither the program (original or migrated) nor the criterion; offer a mutation score as the stronger option (the pilot uses it). (b) Versions are requested only for LLM judges; add versions of tests, compiler or runtime, comparator, mocks and environment. (c) Position the checklist against existing standards (ACM SIGSOFT Empirical Standards; Wagner et al. 2025 guidelines for studies involving LLMs; How2Bench) and say it is an unvalidated proposal; ideally two raters apply it to a sample and report agreement. (d) Oracle provenance is narrower than what the checklist covers (denominators, judge accuracy, limitations): define it or rename it.**]** It counts what the papers report rather than what they did, so it shows how far their reporting is from the checklist and not how good individual studies are. The checklist sets no thresholds for agreement or coverage because suitable values depend on the system being migrated and on the risk of the migration.**[P69, Minor; R3:** Risk is invoked but not connected to oracle choice. A Requires column in Table 1 (runnable original plus inputs for O-f, existing tests for O-c, a formal model for O-g, rater time for O-i, a known-answer set for O-h) and one or two sentences on graded rigor (e.g. integrity levels in IEEE 1012; a JUnit 3 to 4 sweep vs a port of financial calculations) would let a team act on it.**]** Nor does it require a particular oracle: a study that cannot run the original program meets item 6 by stating what prevented it.

8 Threats to Validity

Construct validity. Classifying oracles and contexts requires judgment, for instance when a compiler serves both as a filter and as a reported metric. We mitigate this with a written codebook, by coding only oracles that judge the final output and by discussing hard cases until agreement. Our codes also record what papers write rather than what they do. A paper that states no oracle limitation may still be aware of one, so the 10 papers in that row of Table 10 measure reporting rather than awareness. Short papers describe their evaluation in less detail, so the 7 in the corpus (Section 5.1) may understate the oracles they use. The origin codes in Table 7 likewise depend on what papers describe. Two differential-testing papers do not say how some of their inputs were produced [16, 26], and TransAgent does not say where the expected outputs of its LLM-generated tests come from [60]. Our rule codes TransAgent as O-e; had its expected outputs come from running the original, O-e would count one paper fewer and O-f one more. Other limits come from what we chose to code. We did not code whether tests and comparisons with the original ran in the full environment or with substitutes such as mocks or emulators, so we cannot tell how many results rest on such substitutes. Our legacy flag does not cover C either, so every count for legacy migration depends on that choice (Section 5.3). The gaps in Figure 3 rest on reported numbers, since we re-run no experiment. Each gap also depends on the configuration we plot (named in the figure), and some papers

count outputs that no test covers as not equivalent [25, 61]. The gaps are therefore indicative rather than exact.

Internal validity. A keyword search can miss papers. Of 40 papers that meet our criteria and that we knew beforehand, the string retrieved 39; the scan also found the other 11 included papers. We found these 40 papers in exploratory searches of the same proceedings and in related work. Papers found this way can share the vocabulary of the string, so the check is not fully independent of it. Recall may be lower for papers that describe a migration in other words, like the dependency-repair paper the string missed [15]. Coverage is incomplete in three further ways: for 80 papers without an abstract, the string could be matched against the title only; the poster tracks of FSE 2026 and ASE 2026 had no paper list at the time of the scan; and 3 papers available only behind a paywall were excluded under E6. Screening could exclude a paper on its title and abstract, whereas every inclusion required the full text (Figure 1). An early exclusion therefore risks dropping a relevant paper, and the recall check does not cover this step. Clarification rules decided 36 candidates that the criteria left open (Table 4). Different rules could move some of them across the boundary of the corpus, in particular the 16 that L-TASK excluded because translation was not their main goal. Extraction can misread or overlook details, so we drafted every code from the full text and re-checked every number in the disagreement analysis (Section 5.4) against the papers. Some papers were read in a preprint version, whose numbers may differ from the published one; the screening data mark them.

External validity. The map covers the main proceedings of ICSE, FSE and ASE from 2024 to 2026. Co-located workshops and conferences with their own proceedings are out of scope, although they also cover migration and legacy systems. Journal papers (unless presented at the three conferences) and papers at other venues such as ICSME, SANER and ESEM fall outside the map too.**[P70, Minor; DA, DR:** Also name ML and NLP venues (ACL, EMNLP, NeurIPS, ICLR), where much LLM translation work and many benchmarks appear, probably with more similarity metrics, so the 27% similarity share likely understates the field. One line on the wholesale exclusion of patch backporting and porting papers (C056, C085, C187, C210, C241) would pre-empt a reviewer question, since porting between versions resembles VER.**]** As a result, it misses studies such as the closely related one at ESEM [51]. Two sources of evidence lie outside any map of peer-reviewed venues: preprints on arXiv (where research on LLM-based migration often appears first) and the oracles that companies use in migrations they do not report. The map may lag behind this fast-changing research, and it captures industrial practice only through the 9 papers in industry tracks. Our conclusions therefore describe published practice at the three conferences from 2024 to 2026 and hold most firmly for translation between languages, the subject of 33 of the 49 papers.

Reliability. With 49 papers, we report counts and do not test differences between groups.**[P71, Major; R1, EIC; open since R13, R30, R31:** Reliability still says nothing about who screened and coded or how reliable that was (see the Method comment); replace the deleted calibration paragraph with an honest account and, if one person coded, list it as a threat. If LLM tools assisted screening, extraction, analysis or writing, ACM policy (EASE) requires disclosure, and a paper about trusting LLM verdicts should say how any LLM-assisted coding was checked.**]** Any trend over time rests on three years, and 2026 alone holds more papers than 2024 and 2025 together (27 against 22). A pattern seen only in 2026 may reflect the size of that year rather than a change in practice. Each of the 5 journal-first papers counts under the year in which a conference presented it; dated by journal publication, some could move to an earlier year. The replication package (Section 9) contains our protocol and codebook, recomputes every count from our data**[P72, Minor; R1, EIC:** Overstates until reading_counts.csv is released (your own 30/10 note): several counts come from reading, not from coded fields (see the Directions comment). Narrow it to every count of the coded fields, or add the reading counts.**]** and records the quotations and locations behind the codes of each paper. Our snapshot of the proceedings dates from October 23, 2026.

The pilot. The pilot shows that oracles can disagree on one fixed set of translations, not how often this happens in general or how the oracles would rank elsewhere. With a 7B translator and an 8B judge quantized to 4 bits, it runs once at temperature 0 on 164 single-function problems translated from Python to Java. This setup limits how far the results generalize: a larger judge, another prompt, other language pairs, repository-scale code or several samples per task could change the rates in Table 11. The benchmark adds a second limit, since HumanEval-X has been public for several years and both models may have seen its problems or solutions during training. Such exposure could raise the pass rate and the CodeBLEU scores. A third limit lies in the Java tests shipped with the benchmark, against which we compare the other oracles: they miss up to 16% of the seeded faults that compile (Section 6.1). We neither ran the Python originals on generated inputs (which would give more direct evidence of equivalence) nor checked by hand which side is right when an oracle disagrees with the tests (beyond the examples in Section 6.2). Some translations that pass the tests may therefore be wrong, and some errors we attribute to the other oracles could be the tests' own. A fourth limit is sample size: only 32 translations fail the tests, so the false-accept rates have wide 95% Wilson intervals that overlap. They run from 31% to 64% for compilation, from 26% to 58% for CodeBLEU at 0.6 and from 34% to 66% for the judge. Likewise, the judge leads CodeBLEU at 0.6 by just 5 tasks in agreement with the tests (121 against 116).**[P73, Minor; R1:** All oracles judge the same tasks, so overlap of marginal Wilson intervals is not the right criterion; report the discordant pairs for each pair of oracles with a paired interval (e.g. Newcombe), which stays descriptive. The rates over 164 and 132 and the 41% carry no interval (about 8 points either way at n = 164).**]**

9 Conclusion and Data Availability

We mapped how 49 papers at ICSE, FSE and ASE from 2024 to 2026 decide whether LLM-translated or migrated code behaves like the original. Compilation (35 papers) and existing tests (28) dominate, while only 14 papers run the original program to compare behavior (RQ1). In translation between languages, existing tests come mostly from benchmarks; in library and version migration, they usually come from the project's own suite. None of the 5 papers with a legacy source uses them, and 9 of the 12 papers on library migration

include a human check (RQ2). Where papers report several oracles on the same outputs, the results often disagree. The compile rate exceeds the behavioral rate by more than 10 percentage points in 13 of the 18 papers that give both. In 4 of the 5 papers that place a similarity score next to a pass rate, the score reverses at least one ranking of systems or settings. Weak or few tests are the limitation that papers name most often, and 10 papers state none for their own oracle. Only 3 of the 17 papers with human evaluation report agreement between raters (RQ3).

The pilot shows the same effect when only the oracle varies. Across oracles and thresholds, the share of one fixed set of 164 translations counted as correct ranges from 52% to 90%.**[P74, Major; EIC, R1, DA:** Same issue as in the abstract: give the main-threshold range (67% to 90%, 80% under the tests) and name the setup (a 7B translator, one run).**]** At least one of the three oracles that do not execute the code disagrees with the tests on 46% of the tasks. A small LLM judge accepts half of the translations that fail the tests and rejects 41% of the benchmark's own reference solutions (RQ4). Reported success rates therefore depend on the oracle as well as on the system under evaluation.

Together, these findings form five gaps between current practice and what a migration needs (RQ5)**[P75, Minor; EIC, DA:** Closer to the evidence than the abstract's needs of real migration projects; use one wording in the abstract, RQ5, Section 6 and here.**]**: compile rates and similarity scores diverge from behavioral results (G1); judgments by people or LLMs are rarely validated (G2); test evidence for translation rests largely on benchmarks (G3); some tests are not independent of the system under evaluation (G4); and most studies do not run the original program (G5). Each gap ends with a recommendation for studies that evaluate migration. The checklist for oracle provenance (Table 12) gathers these recommendations and two items outside the gaps for authors to follow and reviewers to check. Questions that better reporting alone cannot answer, such as which oracles catch which faults, are taken up in the directions for further study (Section 7.1).

Data availability. The replication package is available at https://github.com/Nov17th/llm-code-migration-oracles (release **v1.0**).**[P76, Major; EIC:** The tag is a placeholder and the README says there is no license or release yet: create a tagged, licensed release archived with a DOI (e.g. Zenodo). For a double-blind EASE submission, replace this URL (it carries a personal account name) with an anonymized mirror, remove the acknowledgments (they name the course and instructor) and add ACM CCS concepts.**]** For the map, it contains the protocol with its codebook, the search log and the list of scanned papers. It also gives the reason for each screening decision and the data extracted from each included paper, with the evidence for them. A script recomputes the counts of the map from these records. For the pilot, the package contains the code with its prompts, the 164 translations and each oracle's verdict on them. It also includes the outcomes of the checks on the reference solutions and seeded faults. The executed notebook of the run is accompanied by a record of the model digests and dataset hashes.

Acknowledgments. We would like to express our sincere gratitude to Assoc. Prof. Nguyen Thanh Phuong for his valuable guidance, constructive feedback and coordination throughout the Seminar II course.

References

[1] Axel Allain, Aymeric Blot, Djamel Eddine Khelladi, and Mathieu Acher. 2026. Code Transformation Rule Synthesis using LLMs: Potential and Limits. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. ACM, 2639–2651. doi:10.1145/3832783.3837504

[2] Yeaseen Arafat and Stefan Nagy. 2026. PROGnosticator: Testing Source-to-Source Code Translators via Construct-Oriented Fuzzing. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 876–898. doi:10.1145/3797135

[3] Yubo Bai and Tapti Palit. 2025. RustAssure: Differential Symbolic Testing for LLM-Transpiled C-to-Rust Code. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 534–546. doi:10.1109/ase63991.2025.00051

[4] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Vageesh D. C., Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, B. Ashok, and Shashank Shet. 2024. CodePlan: Repository-Level Coding using LLMs and Planning. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 675–698. doi:10.1145/3643757

[5] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. 2015. The Oracle Problem in Software Testing: A Survey. *IEEE Transactions on Software Engineering* 41, 5 (2015), 507–525. doi:10.1109/tse.2014.2372785

[6] Andreas Biesdorf, Eckhard Seibert, Nicole Wengatz, Bart Carpels, and Tom Schoonbaert. 2026. IP-Preserving Enterprise System Modernization via LLM-Driven Specification Extraction and Multi-target Code Generation. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. ACM, 4463–4468. doi:10.1145/3832783.3834533

[7] Venkatesan T. Chakaravarthy, Anamitra Roy Choudhury, Dinesh Garg, Vini Kanvar, Shivmaran S. Pandian, Aditya Raghuvanshi, Yogish Sabharwal, and Amith Singhee. 2026. Enterprise-Scale COBOL-to-Java Translation: LLMs Augmented with Program Analysis. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice*. ACM, 775–785. doi:10.1145/3786583.3786915

[8] Sivajeet Chand, Alexander Pretschner, Steve Haupt, Derui Zhu, and Sushant Kumar Pandey. 2026. DepWareTrans: Dependency-Aware Incremental Repository Migration across Co-executable Languages. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. ACM, 4164–4174. doi:10.1145/3832783.3834505

[9] Xiang Chen, Jiacheng Xue, Xiaofei Xie, Caokai Liang, and Xiaolin Ju. 2025. A Systematic Literature Review on Neural Code Translation. arXiv preprint arXiv:2505.07425. https://arxiv.org/abs/2505.07425

[10] Cristina David, Pascal Kesseli, Daniel Kroening, and Hanliang Zhang. 2025. Quantifying the Benefits of Code Hints for Refactoring Deprecated Java APIs. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. ACM, 444–455. doi:10.1145/3696630.3728567

[11] Simantika Bhattacharjee Dristi and Matthew B. Dwyer. 2026. A Differential Fuzzing-Based Evaluation of Functional Equivalence in LLM-Generated Code Refactorings. In *Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering*. ACM, 1708–1712. doi:10.1145/3803437.3806717

[12] Yali Du, Hui Sun, and Ming Li. 2024. A Joint Learning Model with Variational Interaction for Multilingual Program Translation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. ACM, 1907–1918. doi:10.1145/3691620.3695553

[13] Yali Du, Hui Sun, and Ming Li. 2025. Post-Incorporating Code Structural Knowledge Into Pretrained Models via ICL for Code Translation. *IEEE Transactions on Software Engineering* 51, 11 (2025), 3038–3055. doi:10.1109/tse.2025.3605768

[14] Jia Feng, Wenjie Gan, Cuiyun Gao, Chaozheng Wang, Feng Luo, Xin Xia, Ge Li, and Kui Liu. 2026. Dependency-Guided Repository-Level C-to-Rust Translation with Reinforcement Alignment. In *Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering*. ACM, 954–965. doi:10.1145/3803437.3805266

[15] Lukas Fruntke and Jens Krinke. 2025. Automatically Fixing Dependency Breaking Changes. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 2146–2168. doi:10.1145/3729366

[16] Kavyasri Gajula, Phanindra Kodela, and Sridhar Chimalakonda. 2026. K-COBAgent: A Knowledge-Driven Multi-agent Framework for COBOL Modernization. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. ACM, 4631–4635. doi:10.1145/3832783.3834560

[17] Yi Gao, Xing Hu, Xiaohu Yang, and Xin Xia. 2026. GUIMigrator: Semantics-Preserving Transpilation from Android XML to Compose and SwiftUI. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 4737–4759. doi:10.1145/3808215

[18] Yi Gao, Ziyuan Zhang, Xing Hu, Xiaohu Yang, and Xin Xia. 2026. IntentTester: Intent-Driven Multi-agent Framework for Cross-Library Test Migration. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 1515–1535. doi:10.1145/3797096

[19] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, et al. 2024. The Llama 3 Herd of Models. arXiv preprint arXiv:2407.21783. https://arxiv.org/abs/2407.21783

[20] Lars Gröninger, Beatriz Souza, and Michael Pradel. 2025. ChangeGuard: Validating Code Changes via Pairwise Learning-Guided Execution. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 936–956. doi:10.1145/3715760

[21] Giovani Guizzo, Jie M. Zhang, Federica Sarro, Christoph Treude, and Mark Harman. 2023. Mutation Analysis for Evaluating Code Translation. *Empirical Software Engineering* 29, 1 (2023). doi:10.1007/s10664-023-10385-w

[22] Pranjal Gupta, Pooja Aggarwal, Brent Paulovicks, Prateeti Mohapatra, Rong Lee, and Vadim Sheinin. 2026. Automating Terraform Code Migration through Provider Evolution Knowledge. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering*. ACM, 211–215. doi:10.1145/3786582.3786841

[23] Sandeep Hans, Atul Kumar, Toshiaki Yasue, Kouichi Ono, Saravanan Krishnan, Devika Sondhi, Fumiko Satoh, Gerald Mitchell, Sachin Kumar, and Diptikalyan Saha. 2025. Automated Testing of COBOL to Java Transformation. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. ACM, 227–237. doi:10.1145/3696630.3728548

[24] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, et al. 2024. Qwen2.5-Coder Technical Report. arXiv preprint arXiv:2409.12186. https://arxiv.org/abs/2409.12186

[25] Ali Reza Ibrahimzada, Kaiyao Ke, Mrigank Pawagi, Muhammad Salman Abid, Rangeet Pan, Saurabh Sinha, and Reyhaneh Jabbarvand. 2025. AlphaTrans: A Neuro-Symbolic Compositional Approach for Repository-Level Code Translation and Validation. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 2454–2476. doi:10.1145/3729379

[26] Ali Reza Ibrahimzada, Brandon Paulsen, Daniel Kroening, and Reyhaneh Jabbarvand. 2026. ReCodeAgent: A Multi-agent Workflow for Language-Agnostic Translation and Validation of Large-Scale Repositories. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. ACM, 2409–2421. doi:10.1145/3832783.3837486

[27] Mohayeminul Islam, Ajay Kumar Jha, May Mahmoud, Ildar Akhmetov, and Sarah Nadi. 2025. An Empirical Study of Python Library Migration Using Large Language Models. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 867–879. doi:10.1109/ase63991.2025.00077

[28] Mingsheng Jiao, Tingrui Yu, Xuan Li, Guanjie Qiu, Xiaodong Gu, and Beijun Shen. 2023. On the Evaluation of Neural Code Translation: Taxonomy and Benchmark. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1529–1541. doi:10.1109/ase56229.2023.00114

[29] Miryeong Kang, Wonseok Oh, Gabin An, and Hakjoo Oh. 2026. Pig: Leveraging Large Language Models for Python Library Migrations. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 967–988. doi:10.1145/3797072

[30] Rabimba Karanjai, Sam Blackshear, Lei Xu, and Weidong Shi. 2025. Collaboration Is All You Need: LLM Assisted Safe Code Translation. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. ACM, 671–675. doi:10.1145/3696630.3728521

[31] Xingpei Li, Zhijie Jiang, Zhouyang Jia, Yuanliang Zhang, Zheng Si, Haifang Zhou, and Shanshan Li. 2025. Enhancing Deep Learning Transpilation with LLMs: Automation and Semantic Consistency. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. ACM, 1238–1239. doi:10.1145/3696630.3731431

[32] Runlin Liu, Yuhang Lin, Yunge Hu, Zhe Zhang, and Xiang Gao. 2024. LLM-Based Java Concurrent Program to ArkTS Converter. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. ACM, 2403–2406. doi:10.1145/3691620.3695362

[33] Feng Luo, Jiachen Liu, Cuiyun Gao, Jia Feng, and Kui Liu. 2026. Towards Reliable C-to-Rust Translation with Rule-Guided Reasoning and Reinforcement Learning. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. ACM, 4469–4480. doi:10.1145/3832783.3834534

[34] Yang Luo, Richard Yu, Fajun Zhang, Ling Liang, and Yongqiang Xiong. 2024. Bridging Gaps in LLM Code Translation: Reducing Errors with Call Graphs and Bridged Debuggers. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. ACM, 2448–2449. doi:10.1145/3691620.3695322

[35] Xiangxi Ma, Yifan An, Wentong Tian, Xuanqi Wang, Qingao Dong, Xiang Gao, and Hailong Sun. 2026. DepUpgrade: Automating Dependency Upgrade through State-Path Exploration. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. ACM, 1844–1856. doi:10.1145/3832783.3837442

[36] Marcos Macedo, Yuan Tian, Pengyu Nie, Filipe R. Cogo, and Bram Adams. 2025. INTERTRANS: Leveraging Transitive Intermediate Translations to Enhance LLM-Based Code Translation. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 1153–1164. doi:10.1109/icse55347.2025.00236

[37] Victor May, Diganta Misra, Yanqi Luo, Anjali Sridhar, Justine Gehring, and Silvio Soares Ribeiro. 2026. FreshBrew: A Benchmark for Evaluating AI Agents on Java Code Migration. In *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering*. ACM, 944–955. doi:10.1145/3744916.3773214

[38] Nausheen Mohammed, Akash Lal, Aseem Rastogi, Rahul Sharma, and Subhajit Roy. 2025. LLM Assistance for Memory Safety. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 1717–1728. doi:10.1109/icse55347.2025.00023

[39] Micheline Bénédicte Moumoula, Abdoul Kader Kaboré, Jacques Klein, and Tegawendé F. Bissyandé. 2025. The Struggles of LLMs in Cross-Lingual Code Clone Detection. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 1023–1045. doi:10.1145/3715764

[40] Stoyan Nikolov, Daniele Codecasa, Anna Sjövall, Maxim Tabachnyk, Satish Chandra, Siddharth Taneja, and Celal Ziftci. 2025. How is Google Using AI for Internal Code Migrations?. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 481–492. doi:10.1109/icse-seip66354.2025.00048

[41] Vikram Nitin, Rahul Krishna, Luiz Lemos do Valle, and Baishakhi Ray. 2026. C2SaferRust: Transforming C Projects Into Safer Rust With NeuroSymbolic Techniques. *IEEE Transactions on Software Engineering* 52, 2 (2026), 618–630. doi:10.1109/tse.2025.3641486

[42] Guangsheng Ou, Mingwei Liu, Yuxuan Chen, Yanlin Wang, Xin Peng, and Zibin Zheng. 2025. RustRepoTrans: Repository-level Context Code Translation Benchmark Targeting Rust. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 610–622. doi:10.1109/ase63991.2025.00057

[43] Rangeet Pan, Ali Reza Ibrahimzada, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. 2024. Lost in Translation: A Study of Bugs Introduced by Large Language Models while Translating Code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. ACM, 1–13. doi:10.1145/3597503.3639226

[44] Brent Pappas, Joseph Zalusky, Zachary Burkett, and Paul Gazzillo. 2026. Translation Tag Team: Formal Rules and LLMs Translate More Macros Together Than Apart. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. ACM, 1457–1469. doi:10.1145/3832783.3837412

[45] Kai Petersen, Sairam Vakkalanka, and Ludwik Kuzniarz. 2015. Guidelines for Conducting Systematic Mapping Studies in Software Engineering: An Update. *Information and Software Technology* 64 (2015), 1–18. doi:10.1016/j.infsof.2015.03.007

[46] Daniel Ramos, Inês Lynce, Vasco Manquinho, Ruben Martins, and Claire Le Goues. 2024. BatFix: Repairing Language Model-Based Transpilation. *ACM Transactions on Software Engineering and Methodology* 33, 6 (2024), 1–29. doi:10.1145/3658668

[47] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. CodeBLEU: a Method for Automatic Evaluation of Code Synthesis. arXiv preprint arXiv:2009.10297. https://arxiv.org/abs/2009.10297

[48] Frank Reyes, May Mahmoud, Federico Bono, Sarah Nadi, Benoit Baudry, and Martin Monperrus. 2026. Byam: Fixing Breaking Dependency Updates with Large Language Models. *Empirical Software Engineering* 31, 4 (2026). doi:10.1007/s10664-026-10835-1

[49] Aymane Rgaguena, Imane Chlioui, and Maryam Radgui. 2025. Generative AI for Code Translation: A Systematic Mapping Study. *Engineering Proceedings* 112, 1 (2025), 33. doi:10.3390/engproc2025112033 Presented at ICATH 2025.

[50] Momoko Shiraishi, Yinzhi Cao, and Takahiro Shinagawa. 2026. SmartC2Rust: Iterative, Feedback-Driven C-to-Rust Translation via Large Language Models for Safety and Equivalence. In *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering*. ACM, 980–991. doi:10.1145/3744916.3773259

[51] Hui Sun, Anderson Uchôa, Rohit Gheyi, and Wesley K. G. Assunção. 2026. Evaluating Language Models on Cross-Language Code Functional Equivalence. In *20th International Symposium on Empirical Software Engineering and Measurement (ESEM 2026) (LIPIcs, Vol. 394)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 41:1–41:21. doi:10.4230/LIPIcs.ESEM.2026.41

[52] Takaaki Tateishi and Yasuharu Katsuno. 2026. Translating PL/I Macro Procedures into Java Using Automatic Templatization and Large Language Models. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering*. ACM, 46–50. doi:10.1145/3786582.3786808

[53] Chaofan Wang, Tingrui Yu, Beijun Shen, Jie Wang, Dong Chen, Wenrui Zhang, Yuling Shi, Chen Xie, and Xiaodong Gu. 2026. EvoC2Rust: A Skeleton-guided Framework for Project-Level C-to-Rust Translation. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: Software Engineering in Practice*. ACM, 94–105. doi:10.1145/3786583.3786856

[54] Qilong Wu, Taoran Li, Tianyang Zhou, and Varun Chandrasekaran. 2025. SoK: Understanding (New) Security Issues Across AI4Code Use Cases. arXiv preprint arXiv:2512.18456. https://arxiv.org/abs/2512.18456

[55] Yifan Xia, Chengwei Liu, Zifan Xie, Lyuye Zhang, Peiyu Liu, Kangjie Lu, Yang Liu, Wenhai Wang, and Shouling Ji. 2026. Break to Adapt: Knowledge-Based Updates of Breaking Dependencies in JavaScript. *Proceedings of the ACM on*

Table 12: [new] Checklist for oracle provenance. Gap: the gap in Section 7.1 that the item addresses. In the map: counts from our coding of the main-corpus papers to which the item applies.

	Report	Gap	In the map
1	Each oracle (Table 1) and the behavior it observes, such as return values, state or the rendered screen; the denominator of every rate, such as all outputs or only those that compile**[P67, Major; R3:** Add the equivalence relation that Section 2 says an evaluation must make explicit: input domain (e.g. valid or defined inputs; a Rust panic replacing C undefined behavior is intended), comparator and tolerances (exact, normalized, order-insensitive) and intended deviations. Two rates are as incomparable without it as without their oracle.**]**	—	Not coded; non-compiling outputs left out of the behavioral rate: 2 of the 18 papers in Figure 3
2	A behavioral rate on the same outputs next to every similarity score or result of a well-formedness check such as compilation (O-a, O-b)	G1	O-a: 5 of 13; O-b: 18 of 35
3	Human raters: whether they judge independently and blind to the system, the sample, the rubric and their agreement; LLM judges: the model version, the prompt and the accuracy on known correct and incorrect answers from the evaluated domain	G2	Rater agreement: 3 of 17; judge accuracy: 0 of 2
4	Where the tests and their expected outputs come from (Table 7) and how much of the migrated code the tests cover	G3	Origin not stated: 1 of the 31 papers with O-c, O-d or O-e; coverage not coded
5	Whether the system under evaluation produced the tests or used them to select or repair its outputs, and whether it was trained against similar checks; if so, results on tests that it neither produced nor used	G4	Not coded; results on hidden tests: 1 paper [26]
6	A comparison with the original program on the same inputs, the origin of those inputs and whether the original ran in its full environment; or else what prevented running it	G5	Compared with the original: 14 of 49; inputs not described: 2 of the 14; environment and obstacles not coded
7	The oracle's limitations in the evaluated setting, such as weak tests, unexercised paths, a shared source with the output or ways a system can game it	—	Limitations stated: 39 of 49

Software Engineering 3, FSE (2026), 2442–2464. doi:10.1145/3808116

[56] Xudong Xie, Yuwei Zhang, Wensheng Dou, Yu Gao, Ziyu Cui, Jiansen Song, Rui Yang, and Jun Wei. 2026. RISE: Rule-Driven SQL Dialect Translation via Query Reduction. In *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering*. ACM, 968–979. doi:10.1145/3744916.3773257

[57] Pengyu Xue, Kunwu Zheng, Zhen Yang, Yifei Pei, Linhao Wu, Jiahui Dong, Xiapu Luo, Yan Xiao, Fei Liu, Yuxuan Zhang, Xiran Lyu, Xianhang Li, Xuanyu Zhu, and Chengyi Wang. 2026. TransLibEval: Demystify Large Language Models' Capability in Third-Party Library-Targeted Code Translation. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 920–942. doi:10.1145/3797137

[58] Aidan Z.H. Yang, Yoshiki Takashima, Brandon Paulsen, Josiah Dodds, and Daniel Kroening. 2025. VERT: Polyglot Verified Equivalent Rust Transpilation with Large Language Models. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 1453–1463. doi:10.1109/ase63991.2025.00123

[59] Zhen Yang, Fang Liu, Zhongxing Yu, Jacky Wai Keung, Jia Li, Shuo Liu, Yifan Hong, Xiaoxue Ma, Zhi Jin, and Ge Li. 2024. Exploring and Unleashing the Power of Large Language Models in Automated Code Translation. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1585–1608. doi:10.1145/3660778

[60] Zhiqiang Yuan, Weitong Chen, Hanlin Wang, Xin Peng, Zhenpeng Chen, and Yiling Lou. 2026. TransAgent: Enhancing LLM-Based Code Translation via Fine-Grained Execution Alignment. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 1582–1604. doi:10.1145/3797099

[61] Zhiqiang Yuan, Wenjun Mao, Zhuo Chen, Xiyue Shang, Chong Wang, Yiling Lou, and Xin Peng. 2026. Project-Level C-to-Rust Translation via Pointer Knowledge Graphs. *Proceedings of the ACM on Software Engineering* 3, FSE (2026), 3675–3698. doi:10.1145/3808169

[62] Zhiwang Zhang, Shiting Wen, Zhengkai Yan, Bang Zhu, Chengtao Shen, and Hui Guo. 2026. A Survey of Code-to-Code Translation: From Statistical Rule-Based Systems to LLM Agents. *Algorithms* 19, 10 (2026), 857. doi:10.3390/a19100857

[63] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, et al. 2023. CodeGeeX: A Pre-Trained Model for Code Generation with Multilingual Benchmarking on HumanEval-X. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*. 5673–5684. doi:10.1145/3580305.3599790

[64] Jiaying Zhu, Baoqi Zhang, Mengxia Tao, Kezhi Li, Hao Yan, Qiang Xu, and Min Li. 2026. EquivFusion: Unifying Hardware Equivalence Checking from Algorithms to Netlists via MLIR. In *Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering*. ACM, 102–106. doi:10.1145/3803437.3806400

[65] Ming Zhu, Mohimenul Karim, Ismini Lourentzou, and Daphne Yao. 2024. Semi-Supervised Code Translation Overcoming the Scarcity of Parallel Code Data. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. ACM, 1545–1556. doi:10.1145/3691620.3695524

A Primary Studies

Table 13 lists the 51 included papers, 49 in the main corpus and 2 in the I3 tier, with the coded fields behind the frequency tables and figures of Section 5.**[P77, Minor; R2, DR:** Metadata fixes. MBTA's LLM role should be N (TransCoder and j2py, no LLM). refs.bib: mbta year 2024 (EMSE 29(1), article 19); byam article 113; seven FSE Companion entries (coboljava, codehints, collaboration, deepmodelx, dgir, equivfusion, dristi) carry main-proceedings booktitles; pli and terramod are ICSE-NIER 2026 but not labelled; ICSE 2026 venue strings come in three variants and the publisher field is mixed; the uncited syzygy and manyshot entries have wrong authors (Syzygy: Shetty, Jain, Godbole, Seshia, Sen). All 65 cited works exist and match Crossref or DataCite.**]**

Table 13: Included papers and their coding. Venue: conference, year and track (Res. research, Ind. industry/SEIP, JF journal-first, IVR Ideas, Visions and Reflections). Kind: migration kind (Section 2); A/B means both kinds, and +L marks a legacy source. LLM: role of the LLM (P proposed technique, E evaluated subject, B baseline only, T evaluation tooling, N no LLM); P, E means both roles. Oracles: letters of Table 1. Origin of the ground truth of test-based oracles and of the inputs for O-f (Table 7): c:B benchmark tests, c:P project test suite, c:BP both; d:S translated by the system, d:H translated by hand; e:S expected outputs derived from the specification, e:U not stated; f:F fuzzing, property-based or symbolic inputs, f:T existing tests or data, f:L LLM-generated inputs, f:C curated samples or screens, f:U not described. Stated limitation: theme of Table 10.

Paper	Venue	Kind	LLM	Oracles (O-)	Origin	Stated limitation
AlphaTrans [25]	FSE 2025 Res.	XL	P	b, c, d	c:P, d:S	same source as output
BatFix [46]	ICSE 2025 JF	XL	E	b, c	c:B	weak tests
BDUpdater [55]	FSE 2026 Res.	LIB	P	c, i	c:P	weak tests
Byam [48]	ASE 2026 JF	LIB	P	b, c	c:P	other
C2SaferRust [41]	FSE 2026 JF	XL	P	b, c	c:P	weak tests
CAST [13]	FSE 2026 JF	XL	P, E	a, b, c	c:B	weak tests
COBOL-to-Java testing [23]	FSE 2025 Ind.	XL+L	E	f	f:F	incomplete checks
Code hints [10]	FSE 2025 Ind.	LIB	P	b, f, i	f:F	incomplete checks
CodePlan [4]	FSE 2024 Res.	LIB	P	a, b	–	compiling is not behaving
Collaboration [30]	FSE 2025 IVR	XL	P	a	–	none stated
DeepModelX [31]	FSE 2025 Poster	FW	P	b, c	c:B	none stated
DepTrans [14]	FSE 2026 Ind.	XL	P	a, b, c	c:BP	oracle gaming
DepUpgrade [35]	ASE 2026 Res.	LIB	P	b, h, i	–	compiling is not behaving
DepWareTrans [8]	ASE 2026 Ind.	XL	P	b, c	c:P	weak tests
EvoC2Rust [53]	ICSE 2026 Ind.	XL	P	a, b, c	c:B	weak tests
FreshBrew [37]	ICSE 2026 Res.	VER	E	b, c	c:P	oracle gaming
Fruntke & Krinke [15]	FSE 2025 Res.	LIB	P	b, c, i	c:P	weak tests
Google migrations [40]	ICSE 2025 Ind.	LIB	P	b, c, d, i	c:P, d:S	none stated
GUIMigrator [17]	FSE 2026 Res.	FW	B	a, b, f, i	f:C	misleading similarity
IntentTester [18]	FSE 2026 Res.	LIB/XL	P	b, d, i	d:S	subjective judgment
InterTrans [36]	ICSE 2025 Res.	XL	P	c	c:B	weak tests
IP-preserving modernization [6]	ASE 2026 Ind.	FW/XL+L	P	b, e	e:S	same source as output
Java-to-ArkTS converter [32]	ASE 2024 Demo	XL	P	b, f, i	f:C	none stated
K-COBAgent [16]	ASE 2026 NIER	XL+L	P	f	f:U	same source as output
Lost in Translation [43]	ICSE 2024 Res.	XL	E	b, c, d	c:B, d:H	weak tests
MBTA [21]	ICSE 2024 JF	XL	E	b, c, f	c:B, f:T	weak tests
MSA (Checked C) [38]	ICSE 2025 Res.	DIA	P	a, i	–	subjective judgment
Pig [29]	FSE 2026 Res.	LIB	P	i	–	no executable oracle
PL/I templatization [52]	ICSE 2026 NIER	XL+L	P	i	–	none stated
PROGnosticator [2]	FSE 2026 Res.	VER/XL	T	b, f	f:L	incomplete checks
PtrTrans [61]	FSE 2026 Res.	XL	P	b, f	f:F	weak tests
Python library migration [27]	ASE 2025 Res.	LIB	E	a, b, c, i	c:P	weak tests
ReCodeAgent [26]	ASE 2026 Res.	XL	P	b, c, d, f	c:BP, d:S, d:H, f:L, f:U	weak tests
RISE [56]	ICSE 2026 Res.	DIA	P	b, f, i	f:T	incomplete checks
Rule synthesis [1]	ASE 2026 Res.	LIB/VER	E	a, i	–	no executable oracle
RustAssure [3]	ASE 2025 Res.	XL	P, E	b, d, f, g, i	d:H, f:F	incomplete checks
RustRepoTrans [42]	ASE 2025 Res.	XL	E	b, c	c:P	none stated
Semi-supervised translation [65]	ASE 2024 Res.	XL	B	c	c:B	other
SmartC2Rust [50]	ICSE 2026 Res.	XL	P	b, c, d, f	c:P, d:S, f:T	oracle gaming
TerraMod [22]	ICSE 2026 NIER	LIB	P	a, b, c	c:B	misleading similarity
TransAgent [60]	FSE 2026 Res.	XL	P	a, c, e	c:B, e:U	weak tests
TransGraph [34]	ASE 2024 Poster	XL	P	b, c	c:B	none stated
Translation Tag Team [44]	ASE 2026 Res.	XL	P, E	b, c, i	c:P	subjective judgment
TransLibEval [57]	FSE 2026 Res.	XL	E	b, c, i	c:B	none stated
TRAVEL [33]	ASE 2026 Ind.	XL	P	b, c, f	c:B, f:L	weak tests
UniTrans [59]	FSE 2024 Res.	XL	P, E	a, c	c:B	misleading similarity
VERT [58]	ASE 2025 Res.	XL	P	b, f, g	f:F	incomplete checks
VIM-PT [12]	ASE 2024 Res.	XL	B	a	–	none stated
WCA4Z COBOL-to-Java [7]	ICSE 2026 Ind.	XL+L	P	h	–	none stated
l3 tier: cross-language equivalence detection						
EquivFusion [64]	FSE 2026 Demo	XL	N	g	–	–
Moumoula et al. [39]	FSE 2025 Res.	XL	E	h	–	–